Linux Systems and Performance

PART 1: Linux Execution Model

Week 3: Event Driven I/O and IPC

Sungyong Park, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

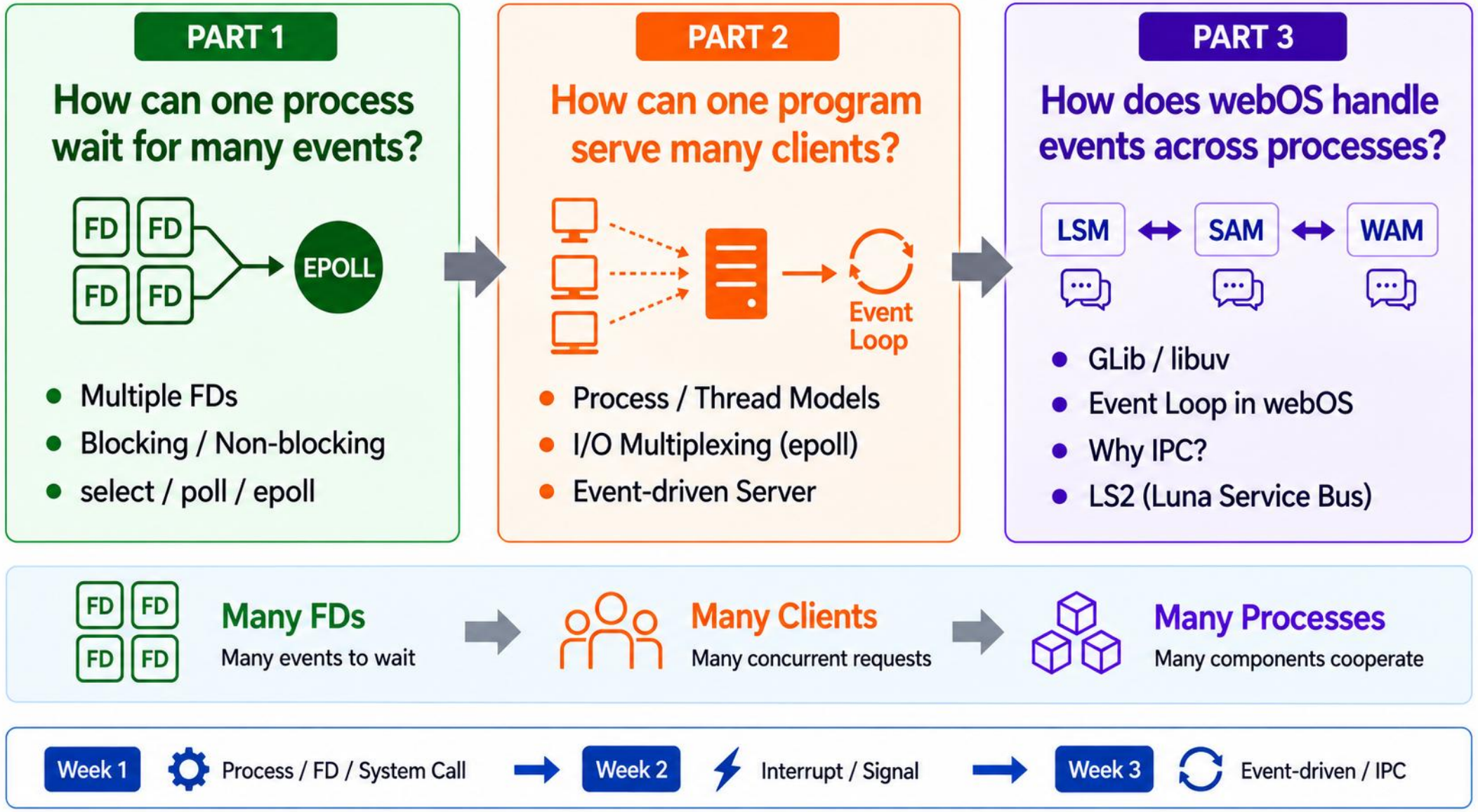
Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr

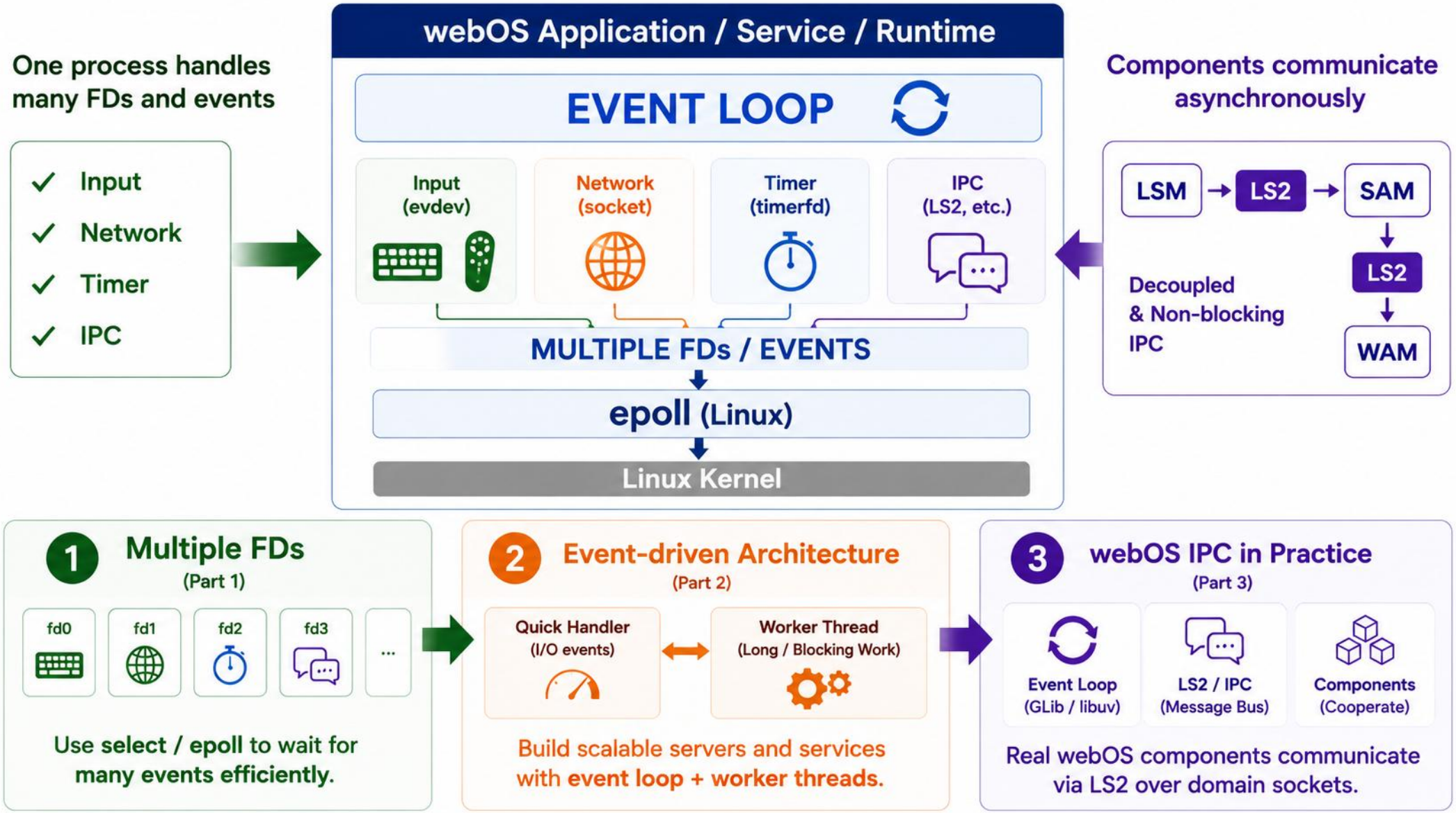
E-mail: parksy@sogang.ac.kr



Roadmap of This Week



How Does This Relate to webOS?



PART 1: Linux Execution Model

- **Week 3**

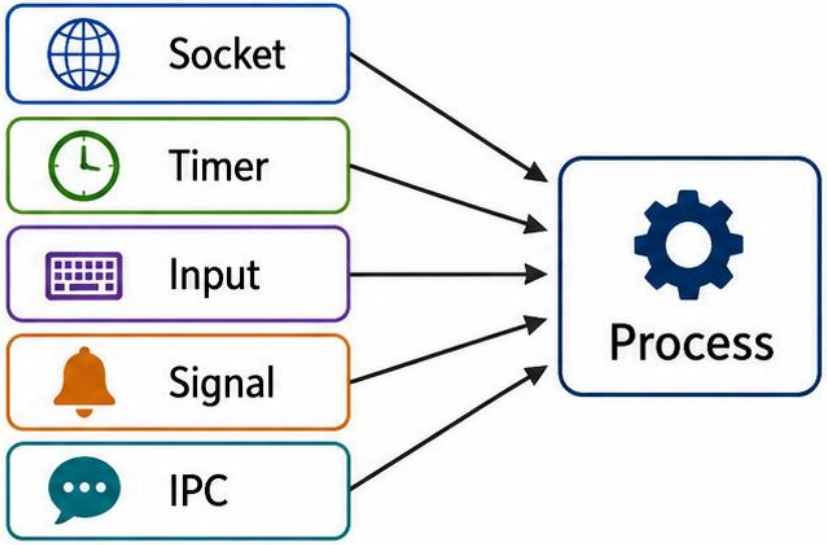
- **How to Handle Multiple Events**
  - How can a process efficiently wait for multiple events?
- **How to Handle Multiple Clients (Concurrent Servers)**
  - How can a server efficiently handle many concurrent clients?
- **Event Driven Applications and IPC in webOS**
  - How can a webOS application handle many events and communicate with other processes efficiently?

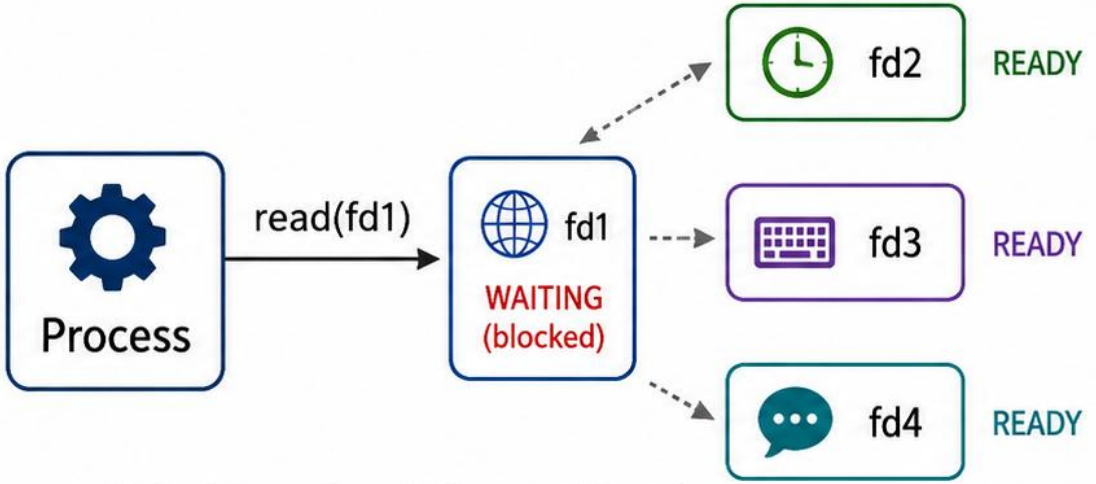
The Problem: Waiting for More Than One Event

A process may need to handle many event sources.

Socket, timer, input, signal, IPC messages, and more may arrive at different times.

How can one process wait efficiently?





A blocking call waits for one FD only.
Other ready events may be delayed.

Requirement

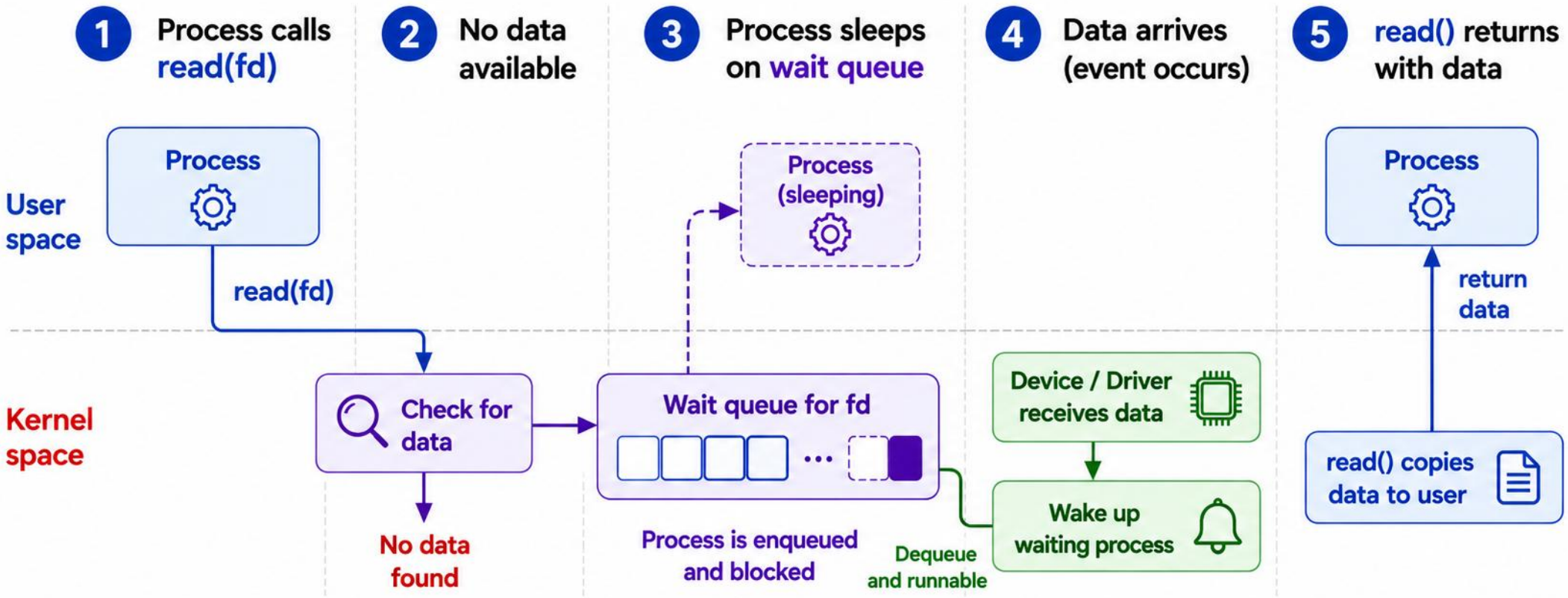
- Wait for many event sources
- Do not waste CPU
- Handle ready events quickly

Goal

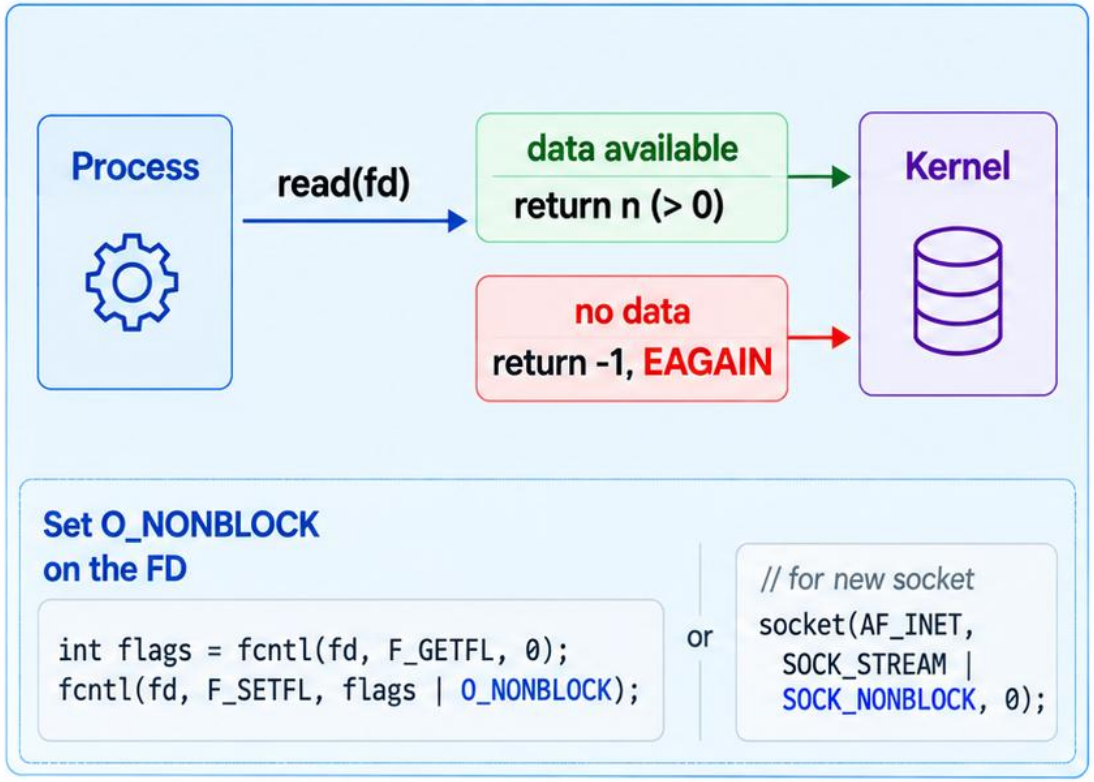
We need a better way to wait for many FDs.

This leads to non-blocking I/O and I/O multiplexing.

What Happens When read() Has No Data?



Non-blocking I/O and Its Limitation




```c
int fds[] = {fd1, fd2, fd3, ...};

while (1) {
  for (int i = 0; i < N; i++) {
    n = read(fds[i], buf, sizeof(buf));
    if (n > 0) {
      handle_data(fds[i], buf, n);
    } else if (n == -1 && errno == EAGAIN) {
      // no data, try next fd
      continue;
    } else {
      handle_error();
    }
  }

  // loop again immediately
}
```

read(fd1) → EAGAIN
read(fd2) → EAGAIN
read(fd3) → EAGAIN
⋮
read(fdN) → EAGAIN

Repeated checking
(busy polling)

Problems of busy polling

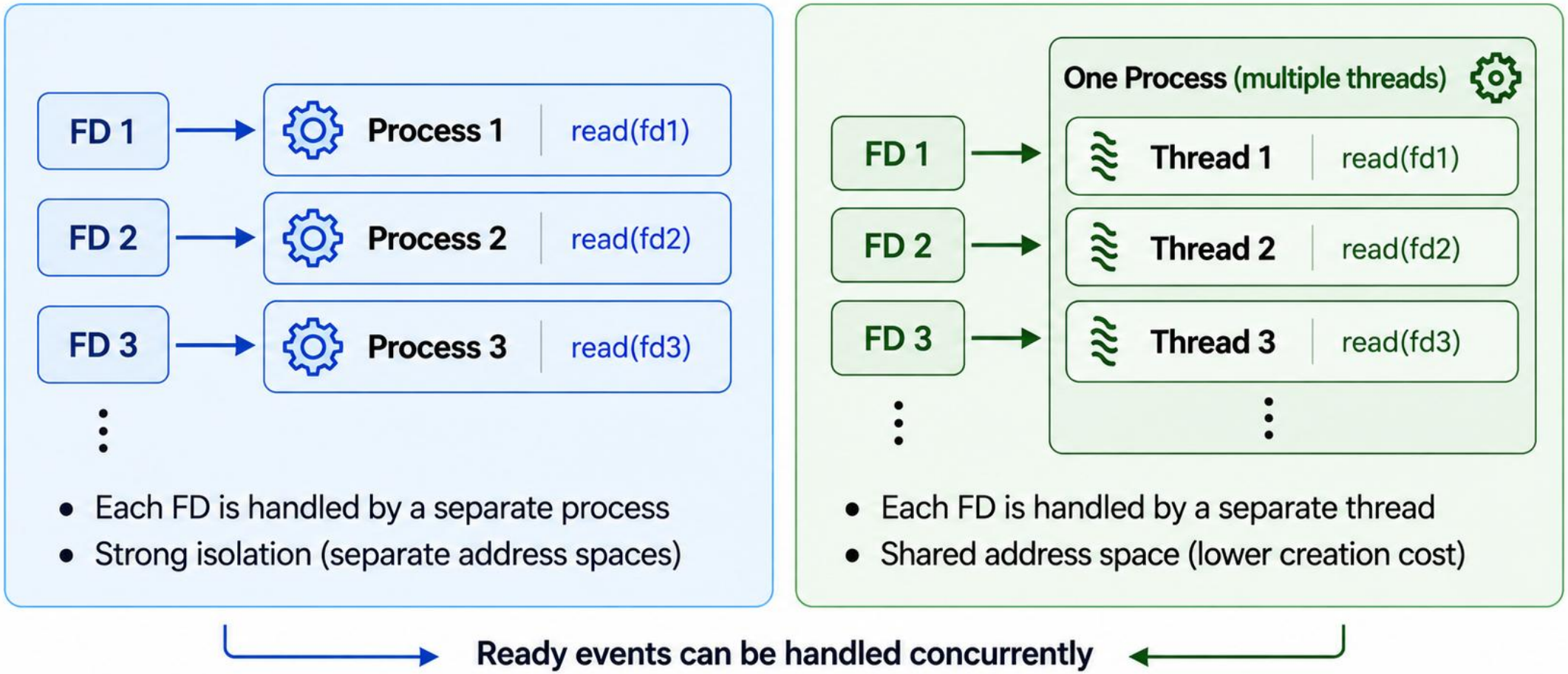
CPU waste
Keeps running even when nothing is ready.

Low efficiency
More checks → more overhead.

Higher latency
Data may be ready, but we detect it later.

Better approach:
Wait until any FD is ready.
(e.g., select, epoll, poll)

Another Solution: One Process / Thread per FD



Dedicate one process or thread to each FD.

This avoids one FD blocking all others, but may require many processes or threads.

Scalability Problem of Process / Thread per FD Solution

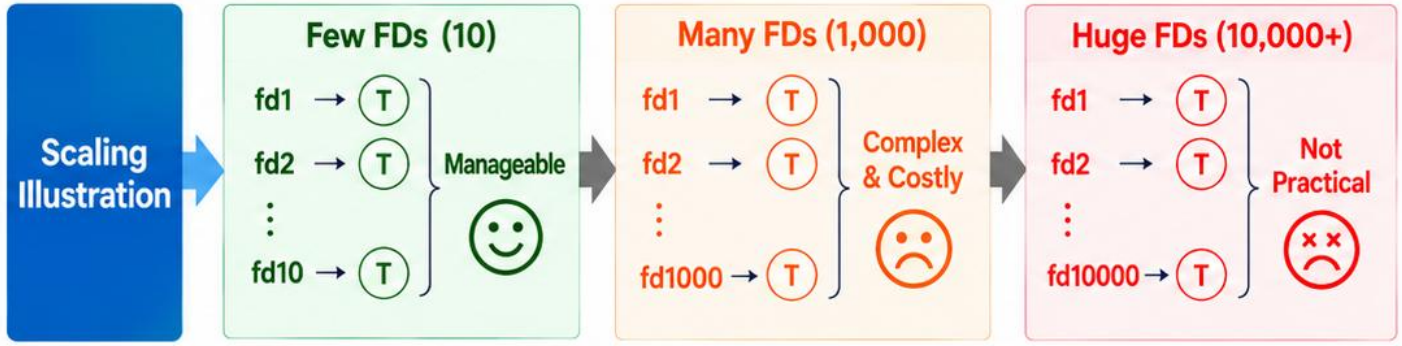
# of FDs	Threads (approx.)	Per-thread Stack (Virtual Address Space)	Context Switch Overhead	Management Complexity
10	10	~ 1 MB (reserved, example)	Low	Low
1,000	1,000	~ 1 MB each (reserved, example)	High	High
10,000	10,000	~ 1 MB each (reserved, example)	Very High	Very High
100,000	100,000	~ 1 MB each (reserved, example)	Extreme	Extreme

Large Stack Address Space
Each thread reserves a stack (e.g., ~1 MB by default, example). Virtual address space grows linearly (e.g., 10K threads ≈ 10 GB reserved).

High Context Switch Overhead
Many runnable threads increase scheduling overhead and reduce CPU efficiency.

Expensive Management
Creating, destroying, and synchronizing many threads is costly and complex.

Synchronization Complexity
Sharing data across many threads requires locks, which can cause contention.



Why This Matters for webOS

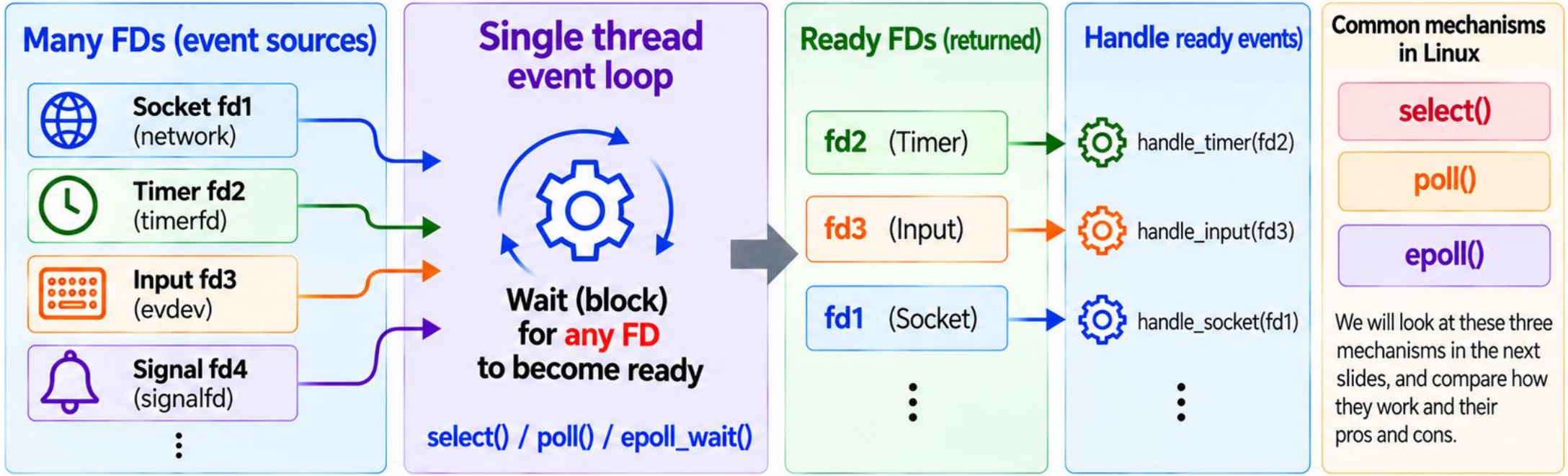


Thread-per-FD does not scale and is not practical for webOS and similar environments.

- Limited memory on devices (e.g., 1–2 GB)
- Shared CPU with real-time workloads (video playback, UI, etc.)
- Many system services already running

Key Takeaway: Each thread reserves a stack (e.g., ~1 MB, example), so thread-per-FD does not scale on memory-constrained systems like webOS. Use event-driven I/O (e.g., epoll/kqueue/io_uring) instead.

Why I/O Multiplexing?



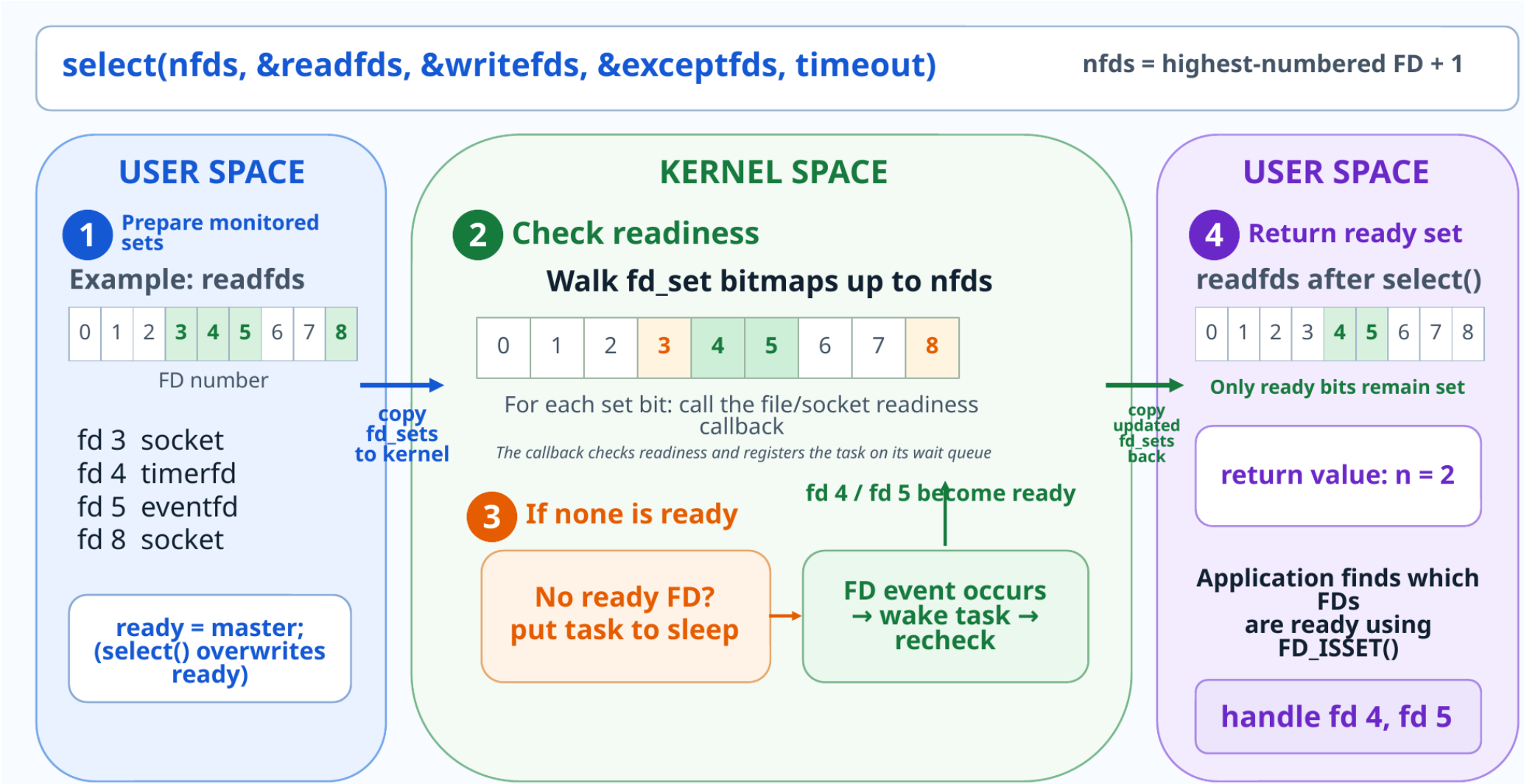
How it works (single-threaded I/O multiplexing)

1. **Register FDs** — Tell the kernel which FDs to watch (socket, timer, input, signal, etc.).
2. **Block in wait** — Kernel sleeps until any event occurs.
3. **Event occurs** — Data arrives / timer expires / input / signal.
4. **Kernel wakes up** — Kernel marks ready FDs.
5. **Return ready FDs** — Kernel returns the list of ready FDs and we handle only the ready ones.

A single thread can achieve high concurrency with low overhead, which is very effective on resource-constrained environments like webOS.

- One thread can handle many I/O sources concurrently.
- **Widely used in real webOS services** (e.g., UI event loop, system daemons, multimedia services).
- Efficient CPU and memory usage, ideal for devices with limited resources.

I/O Multiplexing – select()



AI-generated illustration using ChatGPT (OpenAI)

select() Example

```c
#include <sys/select.h>
#include <unistd.h>

int main(void) {
    int fd1 = ...;   // e.g., stdin or socket
    int fd2 = ...;   // another socket
    int maxfd = (fd1 > fd2) ? fd1 : fd2;

    fd_set master, ready;
    FD_ZERO(&master);
    FD_SET(fd1, &master);
    FD_SET(fd2, &master);

    while (1) {
1       ready = master;   // select() modifies it
2       int n = select(maxfd + 1, &ready,
                       NULL, NULL, NULL);
        if (n < 0) break;
3       if (FD_ISSET(fd1, &ready))
            handle_fd1(fd1);
        if (FD_ISSET(fd2, &ready))
            handle_fd2(fd2);
    }
}
```

1 Before select()

master fd_set	
fd1	1
fd2	1

ready fd_set	
fd1	1
fd2	1

`ready = master`

2 select() blocks

`select(maxfd + 1, &ready, NULL, NULL, NULL);`

returns when at least one FD is ready

3 After select()

ready fd_set	
fd1	0
fd2	1

`n = 1`

`FD_ISSET(fd2, &ready)` → `handle_fd2(fd2)`

master
keeps the monitored set

ready
is copied every loop because select() overwrites it

FD_ISSET()
identifies which FD(s) became ready

select() – Problems and Limitations

1 Fixed-size fd_set

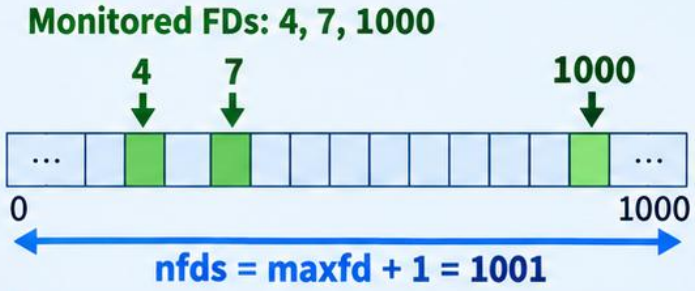
FD_SETSIZE (typically 1024)



FD numbers >= FD_SETSIZE cannot be represented safely.

2 Repeated scanning cost

Cost grows with nfds, not just with the number of ready FDs.

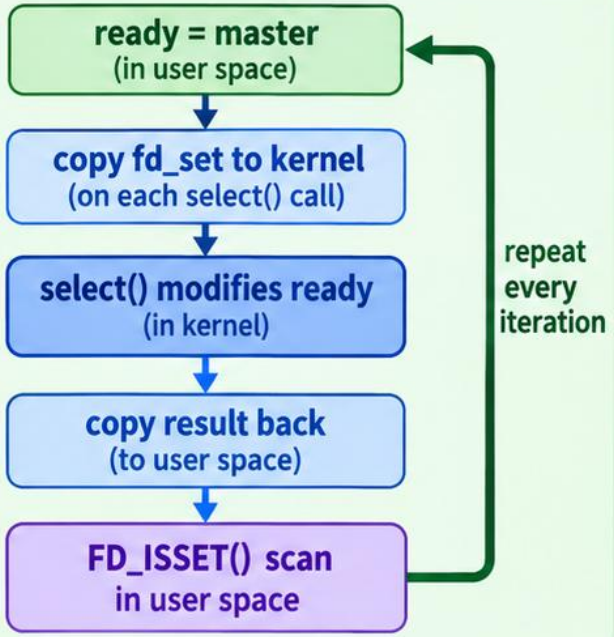


Kernel scans/walks the range (fd_set bitmap) every select() call, even though only 3 FDs are monitored.

N = monitored range (= nfds)
K = ready FDs (e.g., K = 1)

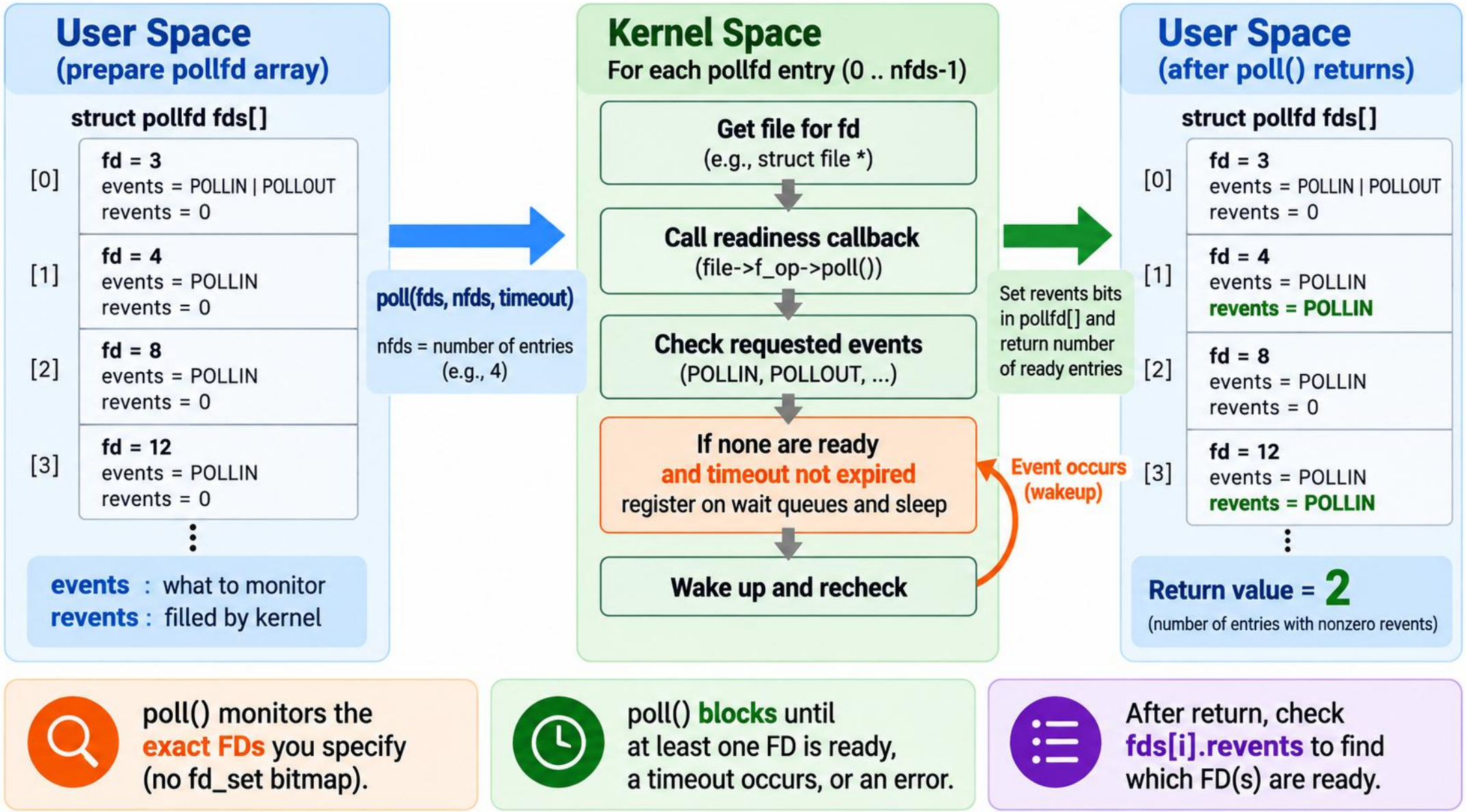
3 Repeated copy + rebuild

No persistent registration in the kernel.



select() is simple, but repeated bitmap copying/scanning and FD_SETSIZE limit scalability. → poll() / epoll()

I/O Multiplexing – poll()



poll() Example

```c
#include <poll.h>
#include <unistd.h>

struct pollfd fds[3];

1  fds[0] = (struct pollfd){ .fd = 3, .events = POLLIN };
   fds[1] = (struct pollfd){ .fd = 4, .events = POLLIN };
   fds[2] = (struct pollfd){ .fd = 8, .events = POLLIN };

while (1) {
2      int n = poll(fds, 3, -1);
       if (n < 0) break;

3      for (int i = 0; i < 3; i++) {
           if (fds[i].revents & POLLIN)
               handle(fds[i].fd);
       }
}
```

1 Before poll(): prepare pollfd[]

index	fd	events	revents (initial)
0	3	POLLIN	0
1	4	POLLIN	0
2	8	POLLIN	0

events = what the application requests

2 poll(fds, 3, -1)

wait for readiness ...

nfds = number of pollfd entries

3 After return: check revents

index	fd	events	revents (from kernel)
0	3	POLLIN	0
1	4	POLLIN	**POLLIN**
2	8	POLLIN	0

revents = what the kernel reports

handle(fd4)

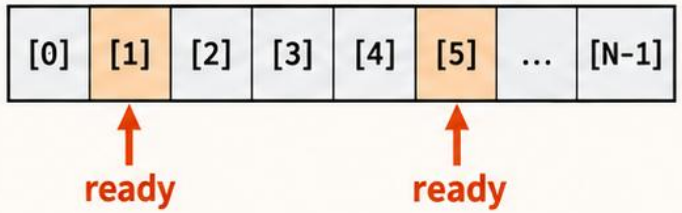
return value: n = 1

poll() checks only the FDs listed in pollfd[]; after return, revents tells which entries are ready.

poll() – Problems and Limitations

1 Linear scan of pollfd[]

pollfd[] (N entries)

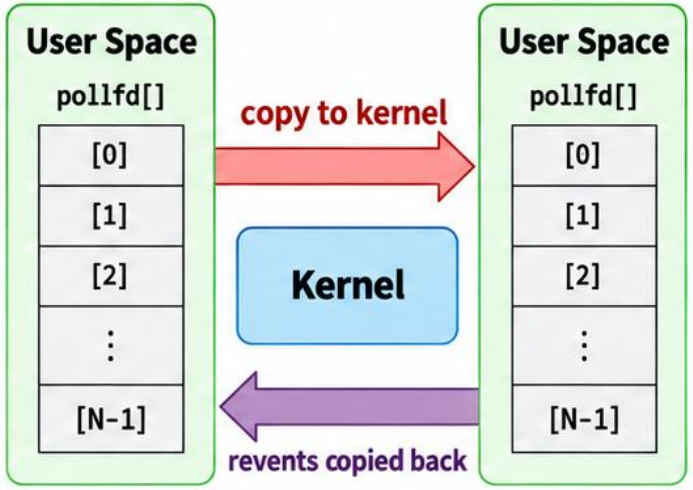


Kernel checks every pollfd entry on each poll() call.

Cost depends on
N = number of monitored FDs.

Even when **K** (ready FDs) is small, **all N entries** are examined.

2 Array copied every call

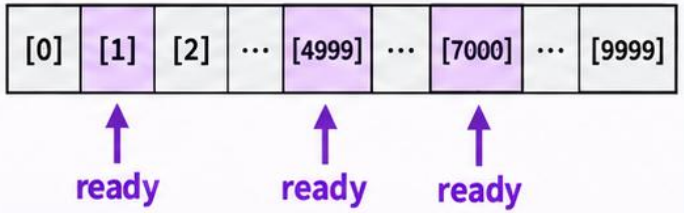


- **No persistent registration.**
- **The same FD list is supplied again on every poll().**

3 User scans results again

```
for (i = 0; i < nfds; i++)
   if (fds[i].revents) handle(fds[i].fd);
```

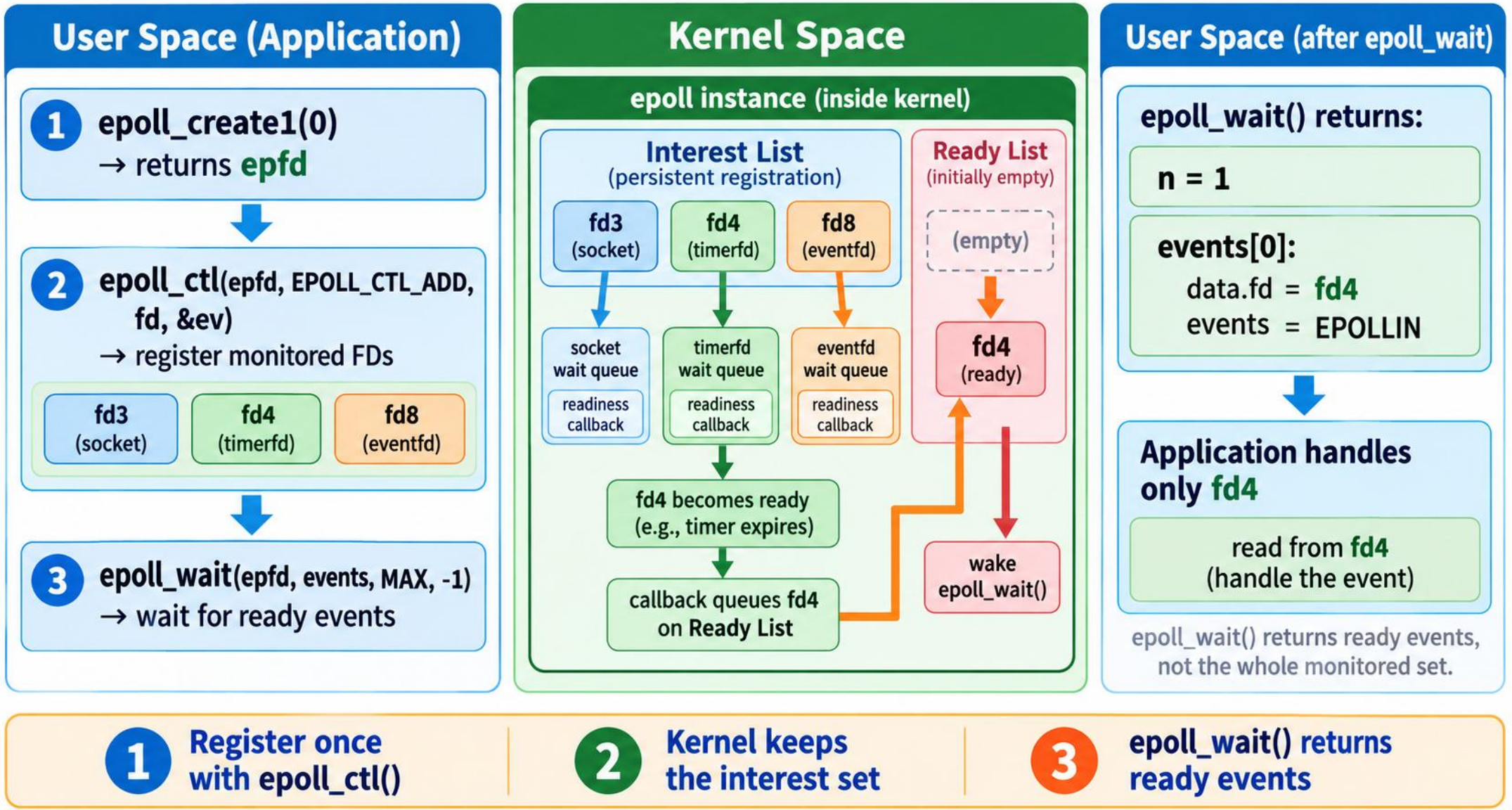
N = 10,000 monitored
K = 3 ready



Application still walks pollfd[] to find ready entries.

poll() removes select()'s FD_SETSIZE / fd-number bitmap problem, but it still does repeated O(N) work each iteration. → **epoll(): register once, wait for ready events.**

I/O Multiplexing – epoll()



epoll() Example

```c
#include <sys/epoll.h>
#include <unistd.h>

int epfd = epoll_create1(0);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = fd4;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd4, &ev);

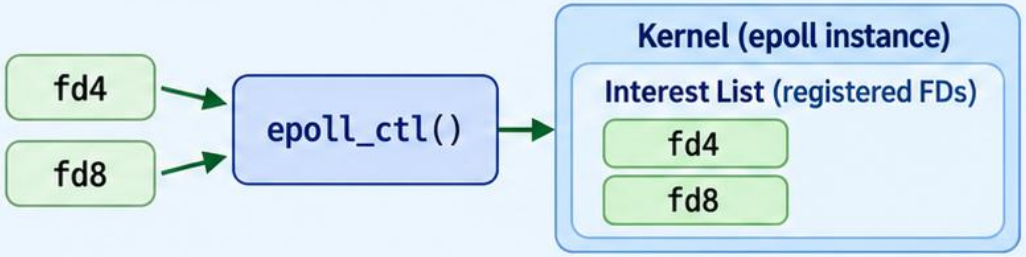
ev.data.fd = fd8;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd8, &ev);

struct epoll_event events[8];

while (1) {
    int n = epoll_wait(epfd, events, 8, -1);
    for (int i = 0; i < n; i++)
        handle(events[i].data.fd);
}
```

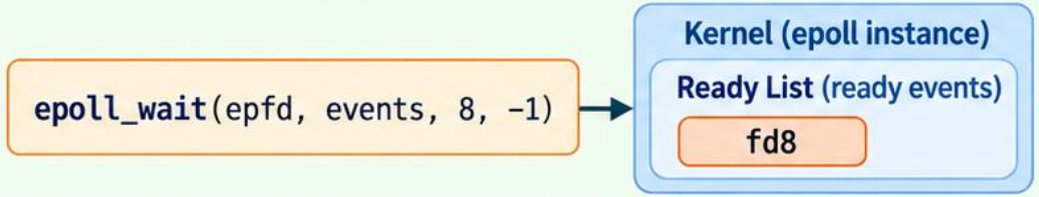
1 Register once

Add file descriptors to the epoll interest list with `epoll_ctl()`.



2 Wait

Block in `epoll_wait()` until one or more FDs are ready.



3 Return ready events

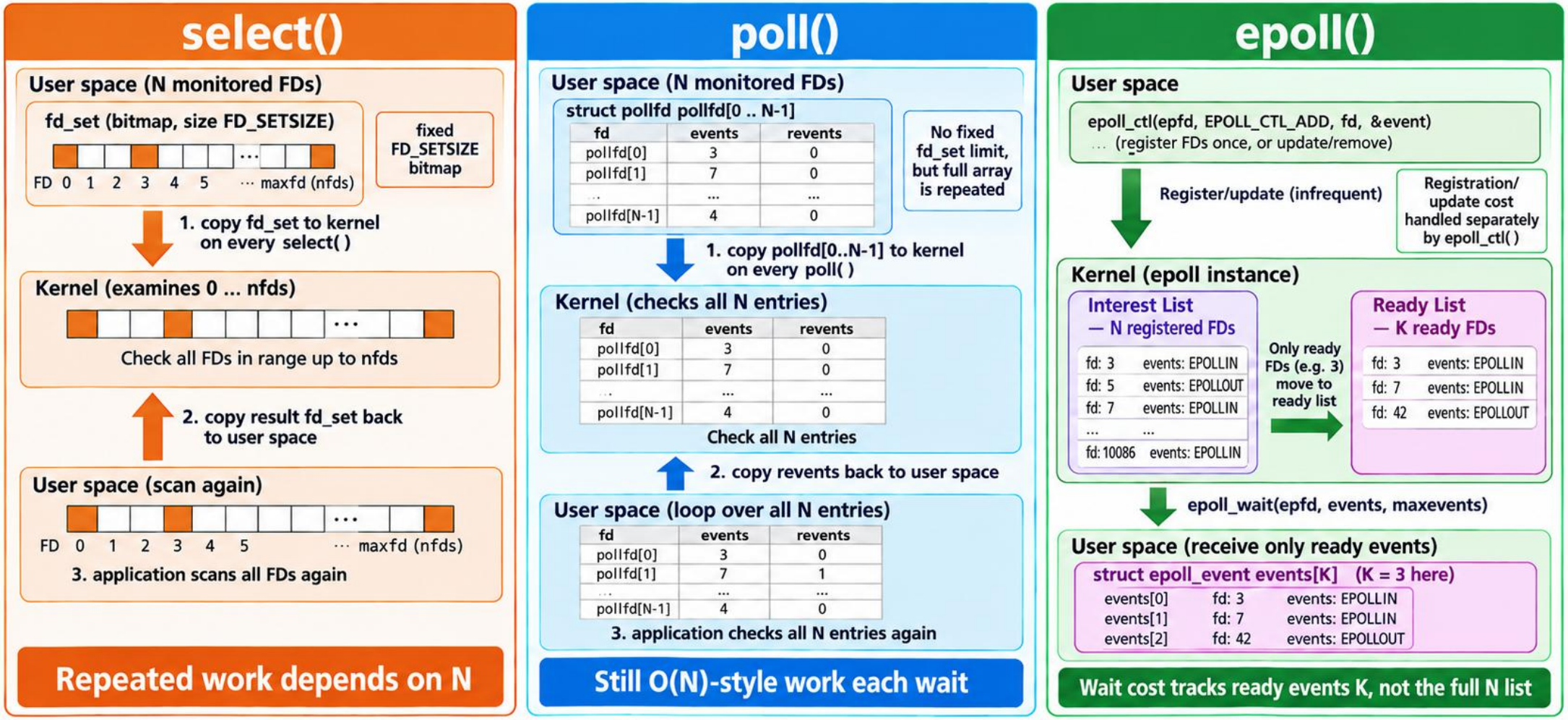
`epoll_wait()` returns only the ready events.



`epoll_ctl()` registers FDs once; `epoll_wait()` returns only the ready events.

No need to resend or rescan the entire monitored FD array on every wait.

epoll() – More Scalable Solution



CHECKPOINT

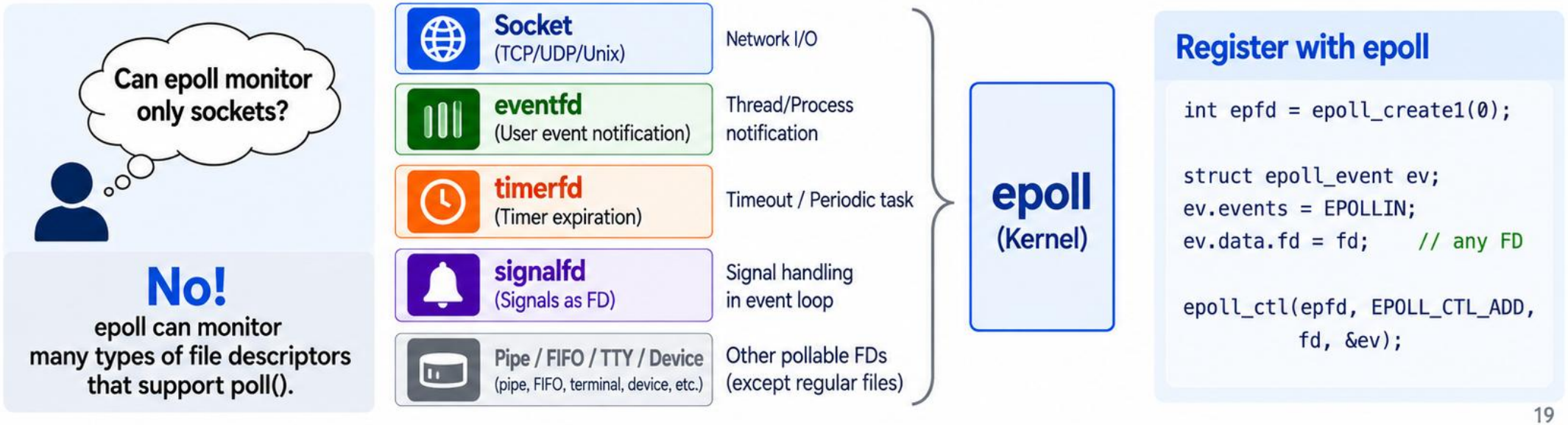
N = 10,000 monitored FDs, K = 3 ready

On each wait, how much work does each perform?

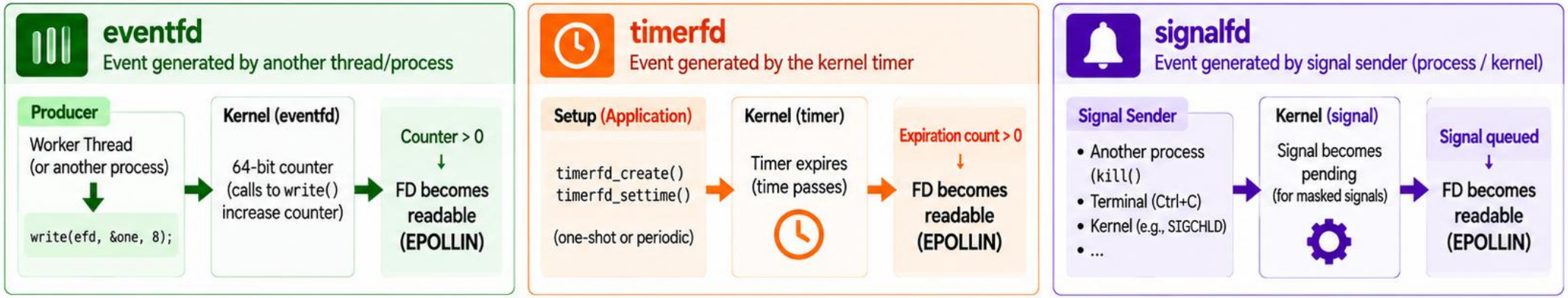
select()	poll()	epoll_wait()
?	?	?

Scalability comes from persistent kernel registration + a ready list — not from rescanning all monitored FDs on every wait.

What FDs Can epoll Monitor?:



19



Regular files (disk files) are not suitable for epoll.

Regular files are usually always ready, or do not support epoll (epoll_ctl may fail with EPERM).
Use epoll for FDs that provide poll() / readiness semantics (e.g., sockets, pipes, devices, eventfd, timerfd, signalfd).

How to Notify Those FDs?

eventfd

Notify from another thread or process

1. Create and register with epoll

```
int efd = eventfd(0, EFD_NONBLOCK);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = efd;
epoll_ctl(epfd, EPOLL_CTL_ADD, efd, &ev);
```

2. Send event (from another thread/process)

```
uint64_t one = 1;
write(efd, &one, sizeof(one));
```

- Increments kernel counter
- Wakes up epoll_wait()

3. Handle the event (in event loop)

```
uint64_t v;
read(efd, &v, sizeof(v));  // consume
```

- v = number of notifications (usually ≥ 1)
- Counter is reset after read

timerfd

Woken by kernel timer expiration

1. Create, set timer, and register

```
int tfd = timerfd_create(CLOCK_MONOTONIC,
                         TFD_NONBLOCK);

struct itimerspec its = {0};
its.it_value.tv_sec = 1;    // first expire in 1s
its.it_interval.tv_sec = 1; // periodic every 1s
timerfd_settime(tfd, 0, &its, NULL);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = tfd;
epoll_ctl(epfd, EPOLL_CTL_ADD, tfd, &ev);
```

2. Event is generated by kernel

- No explicit write() from application
- When the timer expires, kernel increments an expiration count
- tfd becomes readable

3. Handle the event (in event loop)

```
uint64_t expirations;
read(tfd, &expirations, sizeof(expirations));
```

- expirations = number of timer expirations
- may be > 1 if delayed

signalfd

Woken by signal delivery (process / terminal / kernel)

1. Block signals, create and register

```
sigset_t mask;
sigemptyset(&mask);
sigaddset(&mask, SIGTERM);   // handle SIGTERM
sigaddset(&mask, SIGINT);    // handle Ctrl+C
pthread_sigmask(SIG_BLOCK, &mask, NULL);  // block

int sfd = signalfd(-1, &mask, SFD_NONBLOCK);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = sfd;
epoll_ctl(epfd, EPOLL_CTL_ADD, sfd, &ev);
```

2. Send signal (from process / terminal / kernel)

```
kill(pid, SIGTERM);   // from another process
// or press Ctrl+C in terminal (SIGINT)
// or kernel-generated signal (e.g., SIGCHLD, ....)
```

- Signal becomes pending in kernel
- signalfd becomes readable

3. Handle the event (in event loop)

```
struct signalfd_siginfo si;
read(sfd, &si, sizeof(si));
```

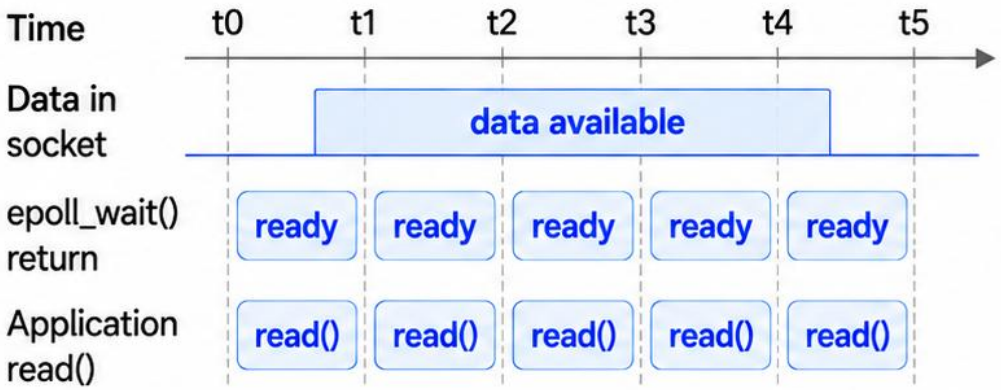
- si.ssi_signo: signal number
- si.ssi_pid: sender pid
- si.ssi_code: reason (e.g., SI_USER, SI_KERNEL, ...)

Level Triggered (LT) vs. Edge Triggered (ET)

Level-triggered (LT)

- If the condition is true (e.g., data is available), the FD is **reported as ready**.
- It will be returned **repeatedly** until the condition is cleared.

Example: socket readable



LT behavior: returned as long as data remains.

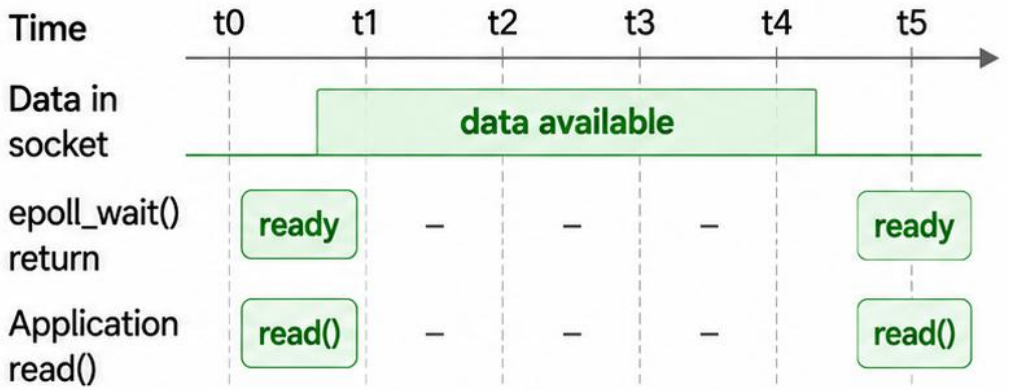
Pros / Cons

- ✓ Easy to use (safer).
- ✗ More wake-ups (returned many times).

Edge-triggered (ET)

- The FD is reported **only when the state changes** (from not-ready to ready, an edge).
- After that, it will **not** be returned again until another edge occurs.

Example: socket readable



ET behavior: returned only on state change (edge).

Pros / Cons

- ✓ Fewer wake-ups (more efficient).
- ✗ Must read until EAGAIN (non-blocking I/O required).

Edge Triggered (ET) + Non-blocking IO

1 Why ET requires non-blocking I/O

ET reports the FD **only once** when it becomes ready (on an edge).

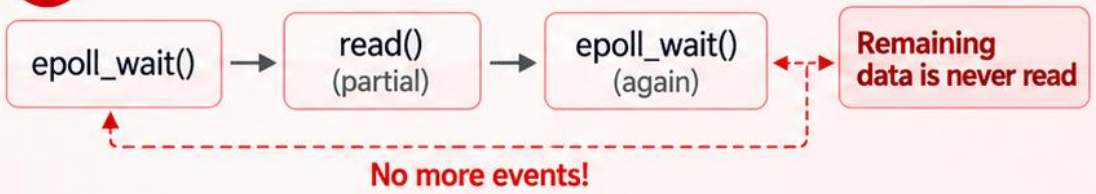
If you read only part of the data and wait in epoll_wait(), there may be **no more notifications** until the next state change.

To drain all available data, the FD must be non-blocking so that read() returns −1 with **EAGAIN** instead of blocking.

→ **Read** (or **write**) in a **loop** until **EAGAIN** (or **EWOULDBLOCK**).

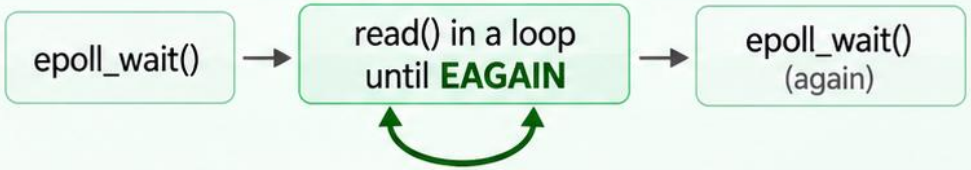
2 What happens if you don't?

Wrong: read only once (partial read)



Result: stalled connection, high latency, or even deadlock.

3 Correct: drain until EAGAIN



Result: all data is consumed, no data left behind.

CHECKPOINT

In **EPOLLET** mode, **4 KB** of data is available in the socket. Your program calls **read()** and consumes only **1 KB**, then returns to **epoll_wait()**.

What can go wrong, and how should it be fixed?

Answer:

1. No new **EPOLLIN** event may occur while unread data remains in the socket buffer (ET reports only on state changes).
2. Use **non-blocking I/O** and keep calling read() in a loop until it returns −1 with **EAGAIN** (or **EWOULDBLOCK**).

Partial Write + EPOLLOUT

read() may return less than requested.

Socket receive buffer



e.g., 1,200 bytes available

`n = read(fd, buf, 4096)`
returns 1200

- read() may return **fewer bytes** than requested.
- Accumulate data until complete.
- Use non-blocking I/O and keep reading until **EAGAIN** (as covered in previous slide).

write() may write only **part** of the buffer.

Application output buffer (per-connection)



Total 4,096 bytes
(3,396 bytes remaining)

write(fd, buf, 4096)

returns 700

Socket send buffer (kernel)



e.g., 700 bytes accepted

1 Partial write occurs

`n = read(fd, buf, 4096)`
returns 700

- Only 700 bytes are accepted by the kernel.
- 3,396 bytes remain.

2 Save progress

- Keep the remaining data in the per-connection output buffer.
- **Update the offset** (e.g., `out_off` = 700).

3 Enable EPOLLOUT

- **Register EPOLLOUT** for this FD.
- When the socket becomes writable again, you will get a notification.

4 Resume writing

- On **EPOLLOUT**, write the remaining data.
- Keep writing until all data is sent or write() returns -1 with **EAGAIN**.
- If done, disable **EPOLLOUT** (to avoid unnecessary wake-ups).

? CHECKPOINT

write(fd, buf, 4096) returns 700.
What should the event loop do next?

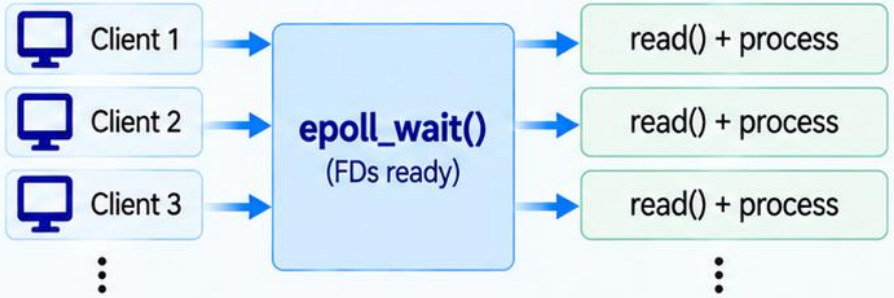
Answer

- Save the progress and keep the remaining **3,396** bytes in the output buffer.
- Enable **EPOLLOUT** for this FD.
- Resume writing when the FD becomes writable again, and keep writing until all data is sent or write() returns -1 with **EAGAIN**.

Backpressure

Normal Situation

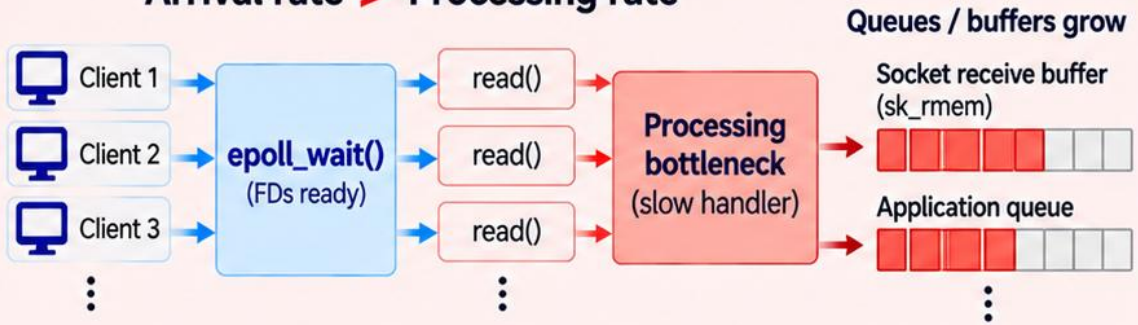
Arrival rate ≤ Processing rate



Buffers stay bounded and latency remains low.

Backpressure Situation

Arrival rate > Processing rate



Data accumulates in kernel buffers and application queues → higher latency.
If sustained, the sender will be throttled by TCP flow control (window reduction).

What epoll CAN do

- Efficiently report which FDs are ready for I/O (readable or writable).
- Avoid scanning all monitored FDs.
- Scale to many connections with low overhead.
- Tell you when you can make progress (on read or write).

What epoll CANNOT do

- Make your handlers run faster.
- Add processing capacity (CPU, threads).
- Provide application-level flow control.
- Prevent backpressure if your application is slower than the incoming data rate.
- Guarantee that all data will be read or written (you must handle partial I/O).

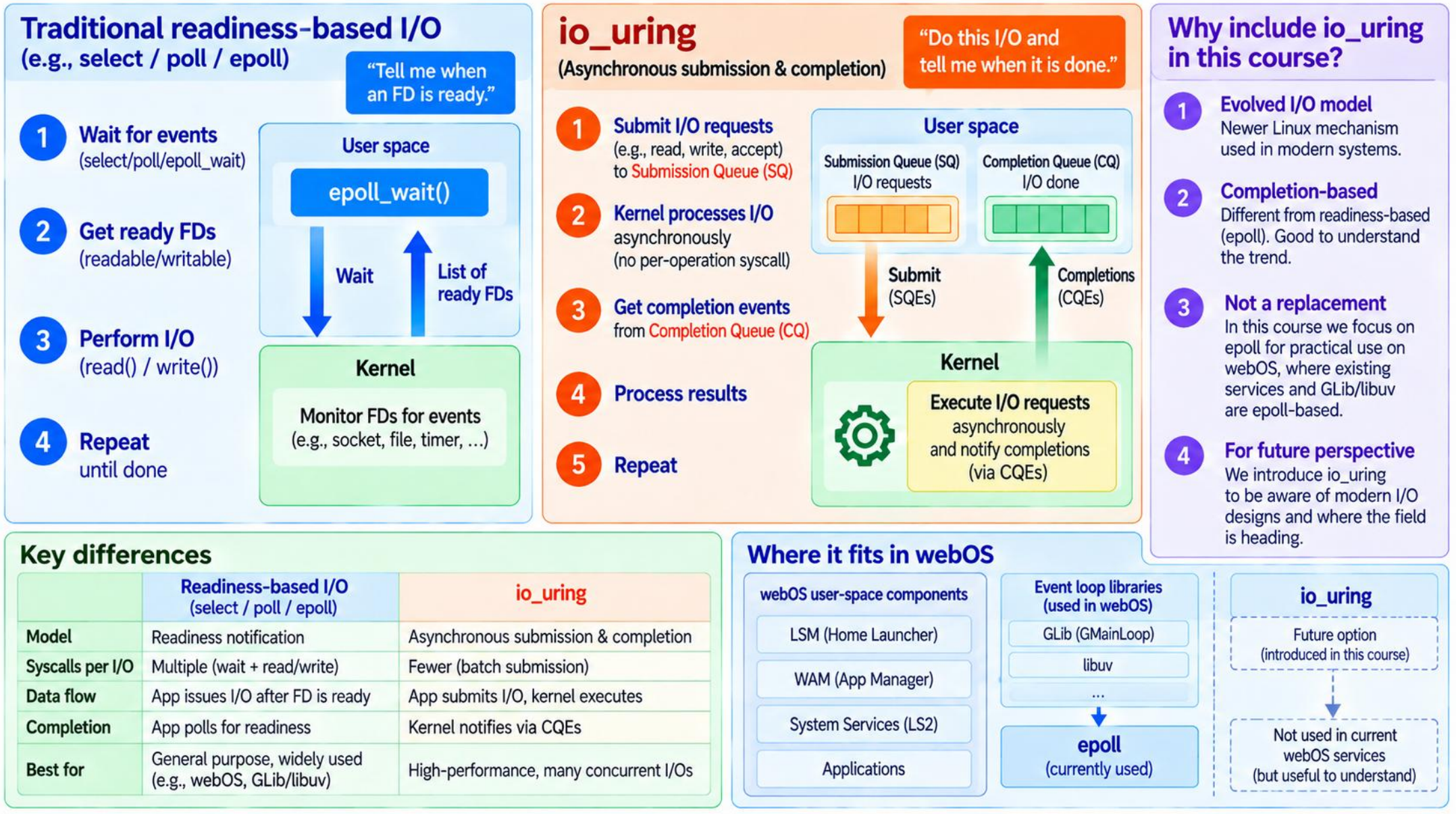
How to handle backpressure

1. Read until EAGAIN to drain available data.
2. Offload slow or CPU-heavy work to worker threads or a thread pool.
3. Bound application queues and buffers (drop, backpressure, or apply admission control).
4. Increase processing capacity or throttle producers (e.g., reduce incoming rate).

epoll scales the I/O readiness notification, but it does not solve backpressure.
Your application must read and process data fast enough and apply proper flow control when needed.

AI-generated illustration using ChatGPT (OpenAI)

From Readiness to Completion: io_uring



	Readiness-based I/O (select / poll / epoll)	io_uring
Model	Readiness notification	Asynchronous submission & completion
Syscalls per I/O	Multiple (wait + read/write)	Fewer (batch submission)
Data flow	App issues I/O after FD is ready	App submits I/O, kernel executes
Completion	App polls for readiness	Kernel notifies via CQEs
Best for	General purpose, widely used (e.g., webOS, GLib/libuv)	High-performance, many concurrent I/Os

Observing epoll and FDs

1 Trace system calls with strace

```
strace -tt -e epoll_wait,epoll_ctl,read,write \
  ./event_loop
```

```
12:00:01.123456  epoll_ctl(3, EPOLL_CTL_ADD, 4, {EPOLLIN, {u32=4}}, 8) = 0
12:00:01.123789  epoll_ctl(3, EPOLL_CTL_ADD, 5, {EPOLLIN, {u32=5}}, 8) = 0
12:00:01.124001  epoll_ctl(3, EPOLL_CTL_ADD, 6, {EPOLLIN, {u32=6}}, 8) = 0
12:00:01.500234  epoll_wait(3, [{events=EPOLLIN, data={u32=5}}], 16, -1) = 1
12:00:01.500300  read(5, "\x01\0\00\\0\0l\0\0", 8) = 8
12:00:01.500345  epoll_wait(3, …, -1) = 0    <timeout>
...
```

- See which FDs are registered (epoll_ctl)
- See when epoll_wait returns and which FD is ready
- See the corresponding read()/write()

Quick way to verify your event loop behavior.

2 Inspect file descriptors

List open FDs of the process

```
ls -l /proc/<pid>/fd
```

```
total 0
lrwx------ 1 user user 64 Sep 10 10:00 0 -> /dev/pts/1
lrwx------ 1 user user 64 Sep 10 10:00 1 -> /dev/pts/1
lrwx------ 1 user user 64 Sep 10 10:00 2 -> /dev/pts/1
lrwx------ 1 user user 64 Sep 10 10:00 3 -> anon_inode:[eventpoll]
lrwx------ 1 user user 64 Sep 10 10:00 4 -> socket:[12345]
lrwx------ 1 user user 64 Sep 10 10:00 5 -> anon_inode:[eventfd]
lrwx------ 1 user user 64 Sep 10 10:00 6 -> anon_inode:[timerfd]
lrwx------ 1 user user 64 Sep 10 10:00 7 -> anon_inode:[signalfd]
```

- Identify the epoll FD and event FDs (eventfd/timerfd/signalfd).
- Useful to check what your program has opened.

See what epoll is monitoring

```
cat /proc/<pid>/fdinfo/<epfd>
```

```
pos: 0
flags: 02000002
mnt_id: 9
ino: 123456
tfd: 4 events: 0x0019 data: 0x00000004
tfd: 5 events: 0x0011 data: 0x00000005
tfd: 6 events: 0x0011 data: 0x00000006
tfd: 7 events: 0x0011 data: 0x00000007
```

- Shows the interest list of the epoll instance.
- Useful for debugging (which FDs are registered and with what events).

3 Use lsof for a higher-level view

```
lsof -p <pid>
```

```
COMMAND     PID  USER   FD     TYPE    DEVICE  SIZE/OFF    NODE  NAME
event_loop  12345  user    3u   a_inode   0,16         0  123456  [eventpoll]
event_loop  12345  user    4u   sock       0,9         0   12345  TCP 127.0.0.1:8080 (LISTEN)
event_loop  12345  user    5u   a_inode   0,16         0  234567  [eventfd]
event_loop  12345  user    6u   a_inode   0,16         0  234568  [timerfd]
event_loop  12345  user    7u   a_inode   0,16         0  234569  [signalfd]
```

- Convenient way to see FD types and targets (sockets, eventfd, timerfd, signalfd, ...).

4 Optional: perf / ftrace for deeper analysis

See where time is spent

```
perf top -p <pid>
# or: perf record -p <pid>; perf report
```

- Identify CPU hotspots in the event loop or handlers (e.g., slow processing).

Trace wake-ups (advanced)

```
sudo trace-cmd record -e sched:sched_wakeup \
  -e sched:sched_switch -p <pid>
```

- Observe when the thread is woken up and how it is scheduled.

PART 1: Linux Execution Model

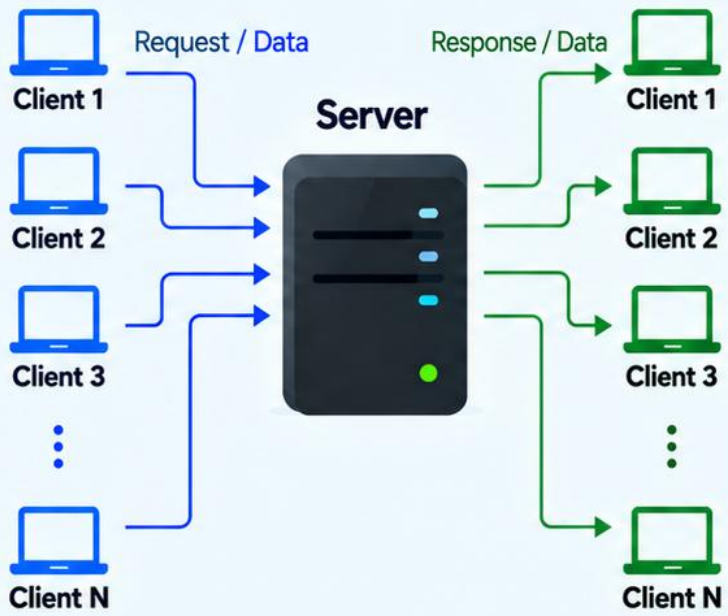
- **Week 3**

- **How to Handle Multiple Events**
  - How can a process efficiently wait for multiple events?
- **How to Handle Multiple Clients (Concurrent Servers)**
  - How can a server efficiently handle many concurrent clients?
- **Event Driven Applications and IPC in webOS**
  - How can a webOS application handle many events and communicate with other processes efficiently?

What is a Concurrent Server?

Many Clients, One Server

A server must handle multiple client connections **at the same time.**



Why a simple blocking server cannot handle many clients?

1. **accept()**
   Get a connection (blocking)
2. **read() / write()**
   Handle client (blocking)
3. **close()**
   Finish client

Blocks while serving one client

waiting...
waiting...
waiting...

Other clients must wait.
A blocking server handles only one client at a time.

What does a concurrent server do?

Overlap I/O and processing
Handle multiple clients without blocking each other.

Use resources efficiently
Keep the server responsive with limited CPU, memory, and threads.

Scale to many connections
Serve thousands of clients (e.g., epoll, event-driven, thread pool).

Key Takeaway

A concurrent server allows many clients to be served efficiently by overlapping their I/O and processing.

- Multiple clients at the same time
- No blocking between clients
- Efficient and scalable

Common Approaches

- I/O multiplexing (select / poll / epoll)
- Thread pool
- Asynchronous I/O (e.g., io_uring, etc.)

Recap: Making a Connection Between Client and Server

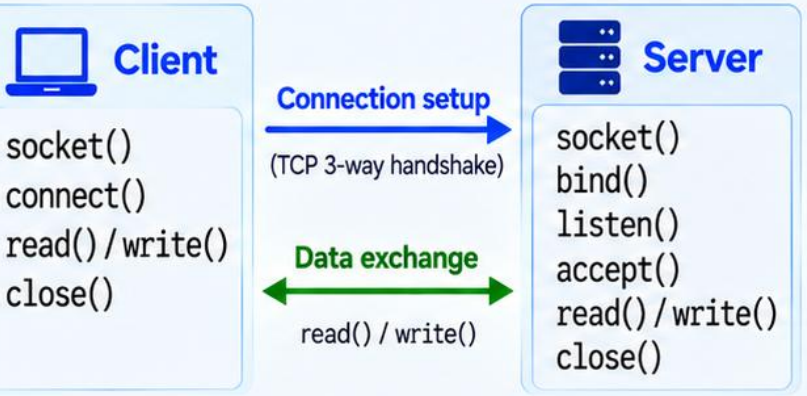
SOCK_STREAM — Connection-oriented

Reliable, ordered byte stream

```
socket(AF_INET, SOCK_STREAM, 0);
```

SOCK_STREAM creates a stream socket.

- Used with **TCP** (AF_INET)
- Uses **connect() / accept()**
- Reliable, in-order, no message boundary (byte stream)
- Widely used for client-server applications (web servers, etc.)



Socket API

- socket()
- bind()
- listen()
- accept()
- connect()
- read() / write()
- close()

Both stream and datagram sockets use a **similar API**.

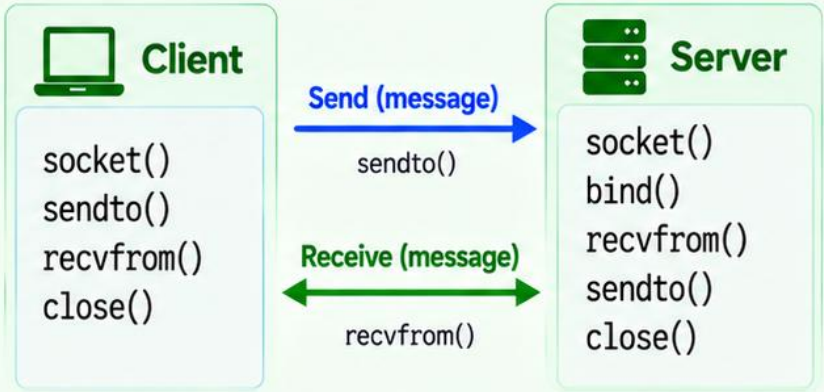
SOCK_DGRAM — Connectionless

Message-oriented, not reliable

```
socket(AF_INET, SOCK_DGRAM, 0);
```

SOCK_DGRAM creates a datagram socket.

- Used with **UDP** (AF_INET)
- **No connect() / accept()** needed (typically)
- Each **send()** is one message (message boundary)
- Not reliable (no delivery/order guarantee)
- Useful for simple, lightweight communication (e.g., DNS, syslog, real-time data)





Both stream and datagram sockets use a **similar API**.
In **webOS**, IPC mechanisms mainly use **stream sockets (SOCK_STREAM)**.

Recap: Listening FD and Connected FDs in Stream Socket

1 listenfd (listening socket)

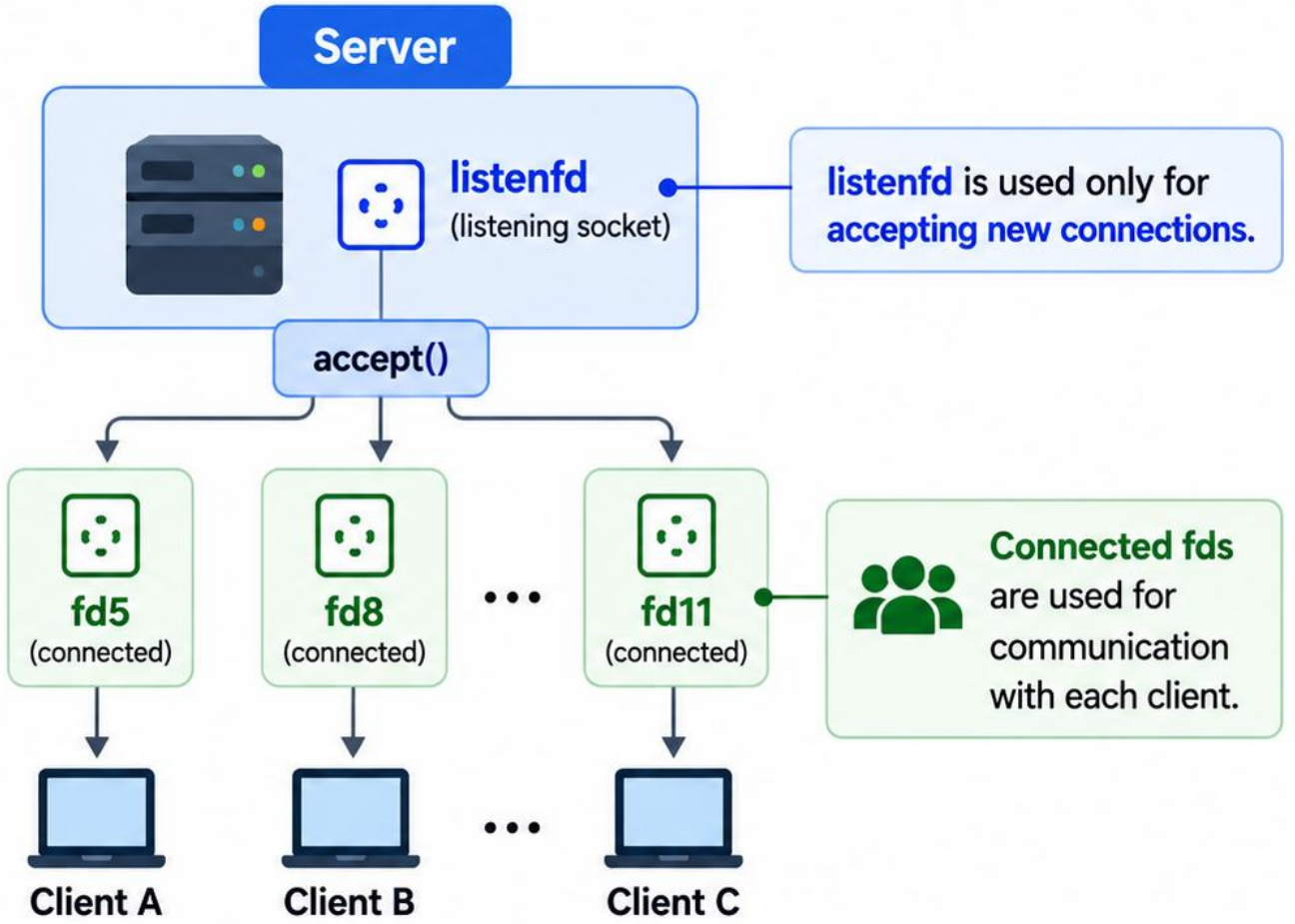
- Created by **socket()** + **bind()** + **listen()**
- Stays in the server and waits for **new connections**.

2 accept()

- When a client connects, accept() returns a **new connected socket** (fd).

3 Each client has its own fd.

- Each connected fd is used for read() / write() with that client.



Key Idea:
One **listenfd** accepts, many **connected fds** talk.

Unix Domain Socket (UDS)

TCP Socket (AF_INET)

Network communication using **IP address** and **port**

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
```

Address family: **AF_INET** (IPv4 / IPv6)

Socket type: **SOCK_STREAM** (connection-oriented)



- Uses **IP address** and **port**
- Supports SOCK_STREAM (TCP) and SOCK_DGRAM (UDP)
- Communicates over the network (can reach remote hosts)

socket() parameters

```
int socket(int domain,
           int type,
           int protocol);
```

domain	Address family (e.g., AF_INET, AF_UNIX)
type	Communication type (e.g., SOCK_STREAM, SOCK_DGRAM)
protocol	Usually 0 (let the system choose)

Both TCP and Unix domain sockets use a **similar socket API.**

Unix Domain Socket (AF_UNIX)

Local communication on the **same host**

```
int fd = socket(AF_UNIX, SOCK_STREAM, 0);
```

Address family: **AF_UNIX** (local)

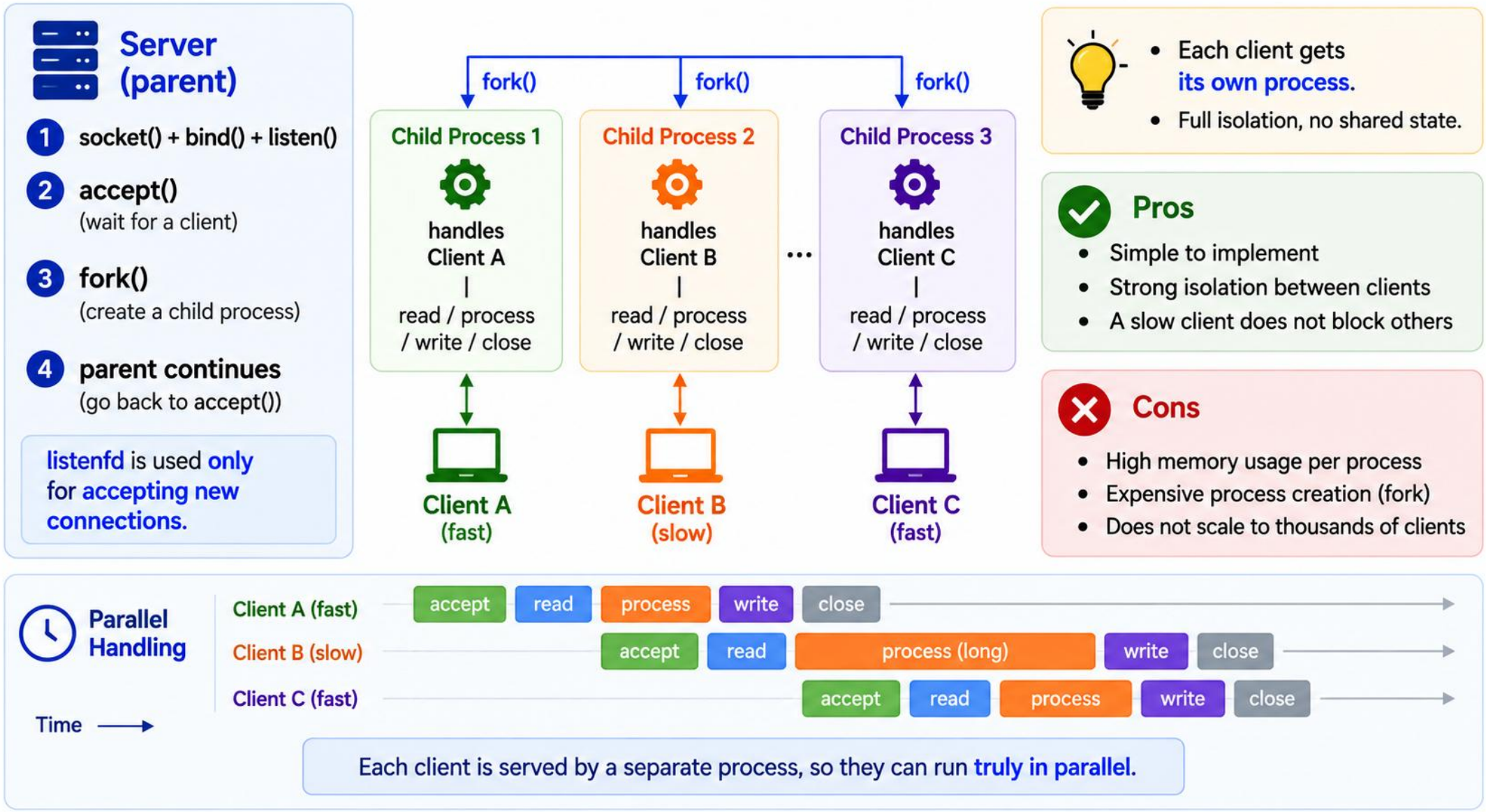
Socket type: **SOCK_STREAM** (connection-oriented)



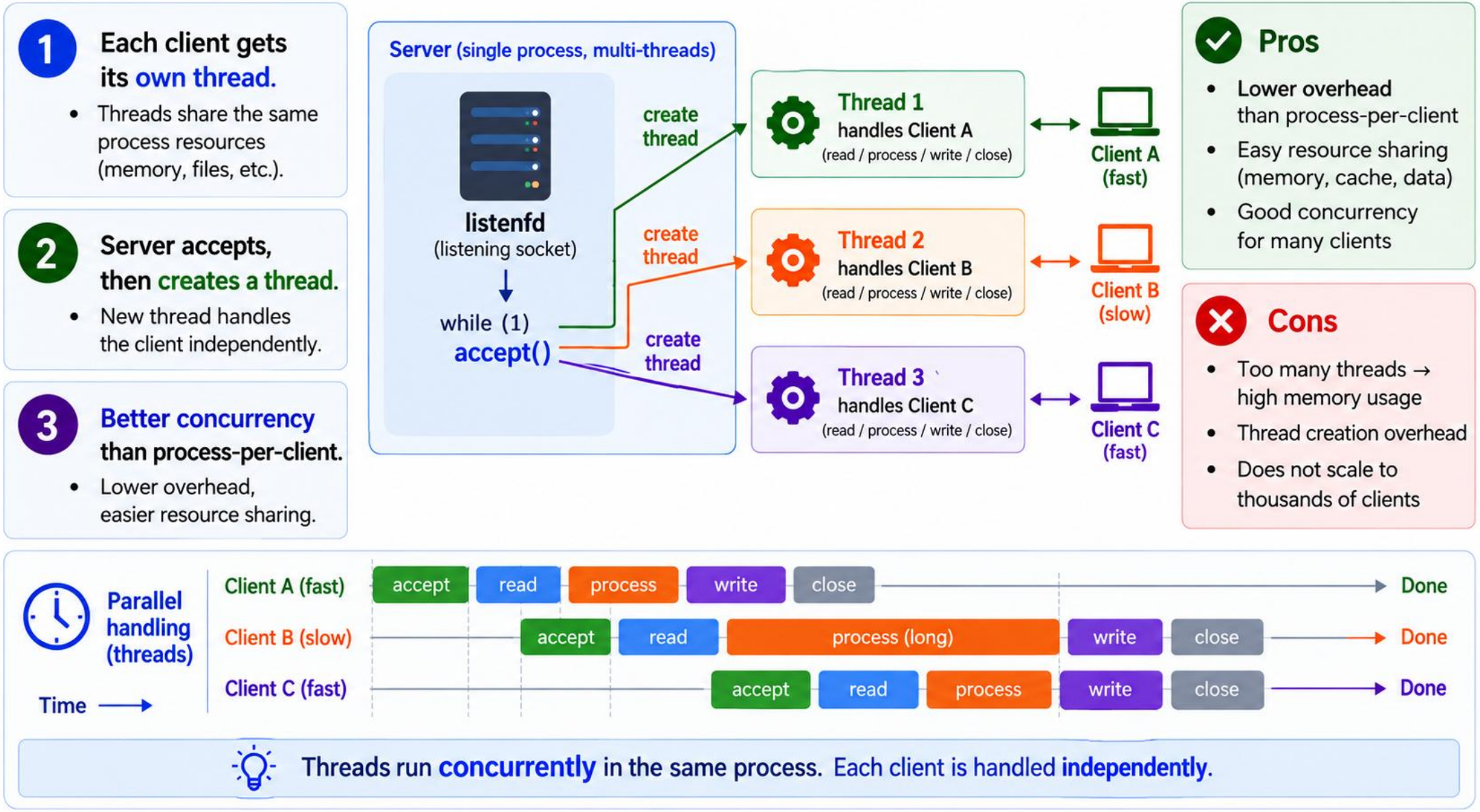
- Uses a file system path (e.g., /tmp/service.sock)
- Supports SOCK_STREAM and SOCK_DGRAM
- No IP, no network stack (local only)
- Typically lower overhead and faster

Both TCP and Unix domain sockets use a **similar socket API.** In **webOS**, IPC mechanisms mainly use **stream sockets (SOCK_STREAM)** over Unix domain sockets.

Solution 1: Process-per-Client Server



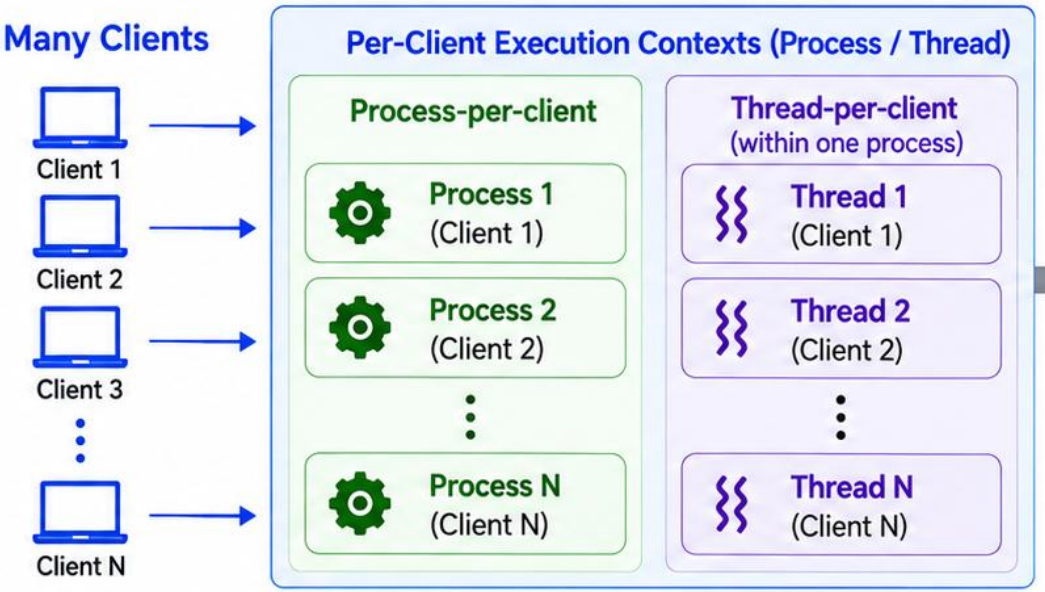
Solution 2: Thread-per-Client Server



Why Per-Client Execution Does Not Scale

Process-per-client or Thread-per-client does not scale.

As the number of clients grows, resource usage and overhead explode.



Explosive Growth

- Processes / threads increase linearly with clients.
- Eventually system becomes overloaded.

Why it does not scale

High Memory Usage
Each process has its own address space. Threads also consume memory (stack, TCB).

Expensive Creation & Destruction
fork()/exec() or thread creation has non-trivial cost.

Context Switch Overhead
More processes/threads → more context switches → higher CPU overhead.

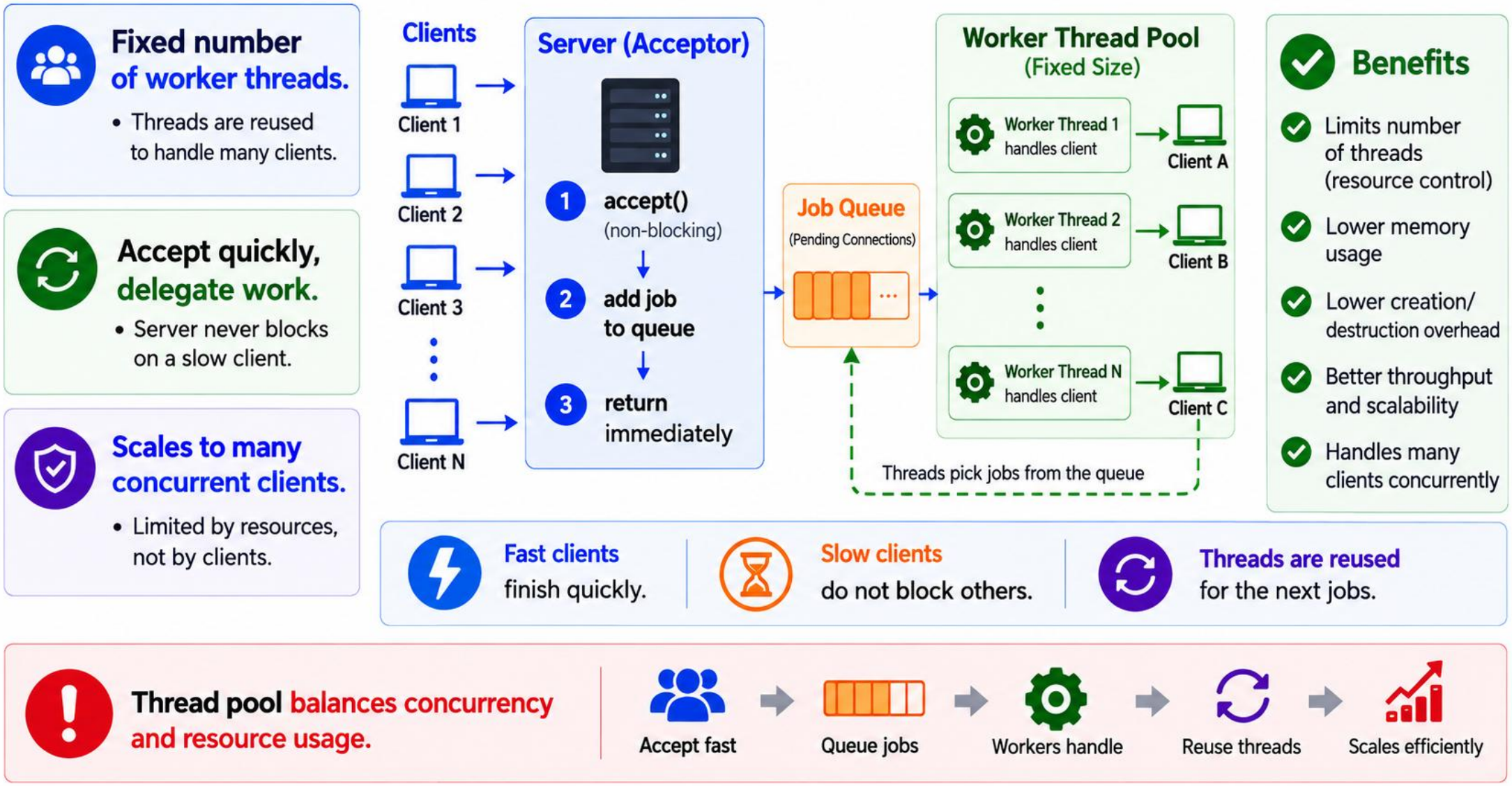
Synchronization Cost
Threads require locks, atomics, and other synchronization.

Resource Limits
Limited by PID/Thread limits, memory, file descriptors, etc.

These models work for small loads, but fail under high concurrency.

We need a better approach that scales.

Thread Pool


Fixed number of worker threads.
• Threads are reused to handle many clients.
Accept quickly, delegate work.
• Server never blocks on a slow client.
Scales to many concurrent clients.
• Limited by resources, not by clients.
Clients
Client 1
Client 2
Client 3
Client N
Server (Acceptor)
1 accept() (non-blocking)
2 add job to queue
3 return immediately
Job Queue
(Pending Connections)
Worker Thread Pool
(Fixed Size)
Worker Thread 1 handles client
Worker Thread 2 handles client
Worker Thread N handles client
Client A
Client B
Client C
Threads pick jobs from the queue
Benefits
Limits number of threads (resource control)
Lower memory usage
Lower creation/ destruction overhead
Better throughput and scalability
Handles many clients concurrently
Fast clients finish quickly.
Slow clients do not block others.
Threads are reused for the next jobs.
Thread pool balances concurrency and resource usage.
Accept fast
Queue jobs
Workers handle
Reuse threads
Scales efficiently

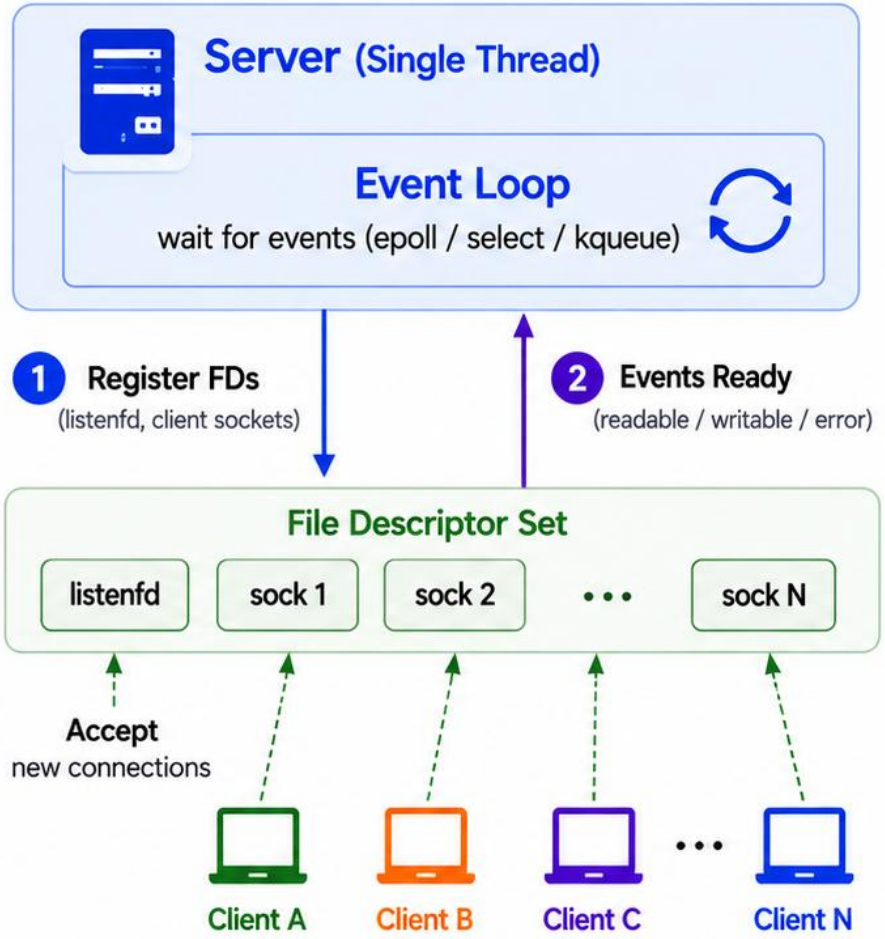
Solution 3: I/O Multiplexing-Based Server

One thread handles many clients.
- No per-client thread/process.

Event-driven I/O multiplexing.
- Wait for events on many FDs simultaneously.

Very efficient resource usage.
- Low memory, low context switching.

Scales to thousands of clients.
- Limited mainly by hardware and implementation.



Event Loop Flow

1. `epoll_wait()` / `select()` / `kqueue()` — Block until an event occurs on any FD
2. Get ready events — Which socket is ready?
3. Handle the event
   - `accept()` new connection
   - `read()` data
   - `write()` response
   - `close()` connection
4. Go back to wait — Repeat forever

Event Types

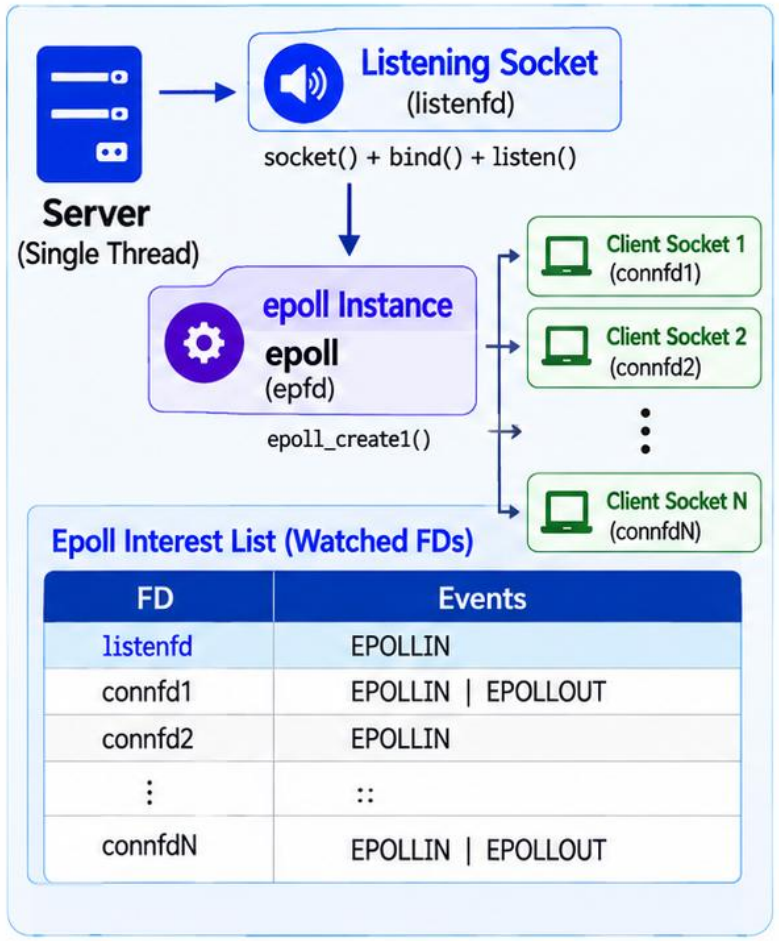
Readable	Data available to read
Writable	Ready to write (non-blocking)
Error / Hangup	Connection closed or error

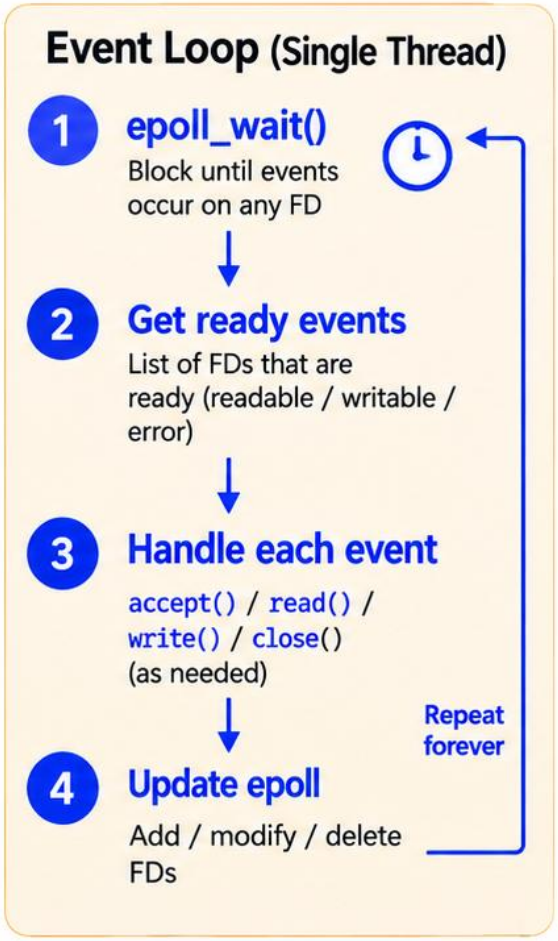
The server **never blocks** on a single client.
It reacts to events and serves active clients **efficiently**.

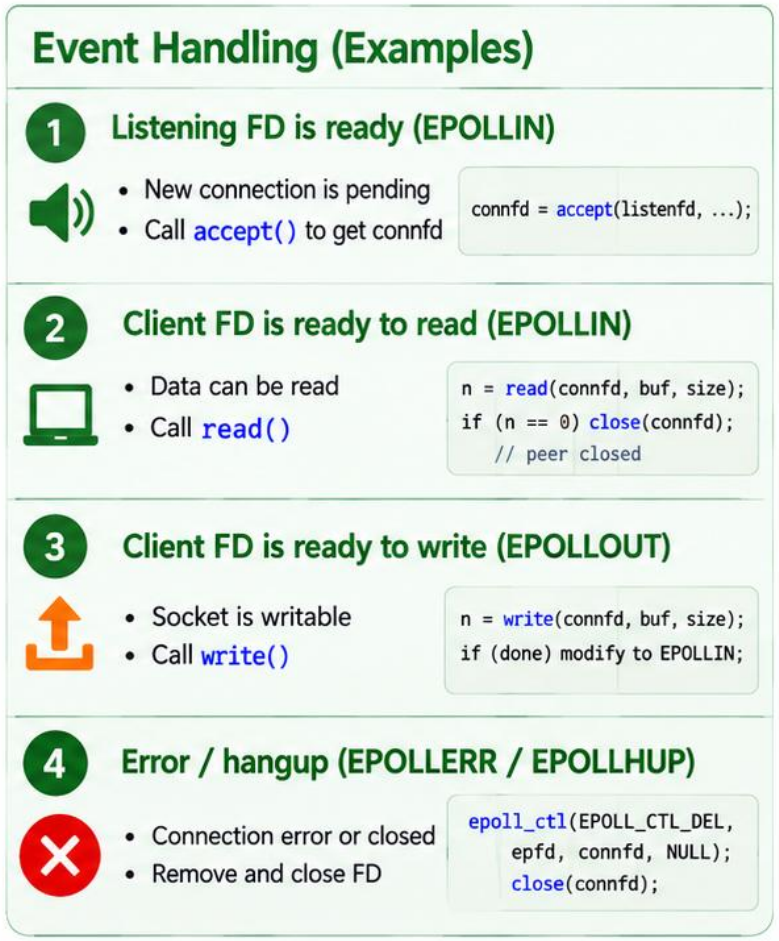
→ **High concurrency** with minimal resources.

AI-generated illustration using ChatGPT (OpenAI)

Minimal epoll-Based Server

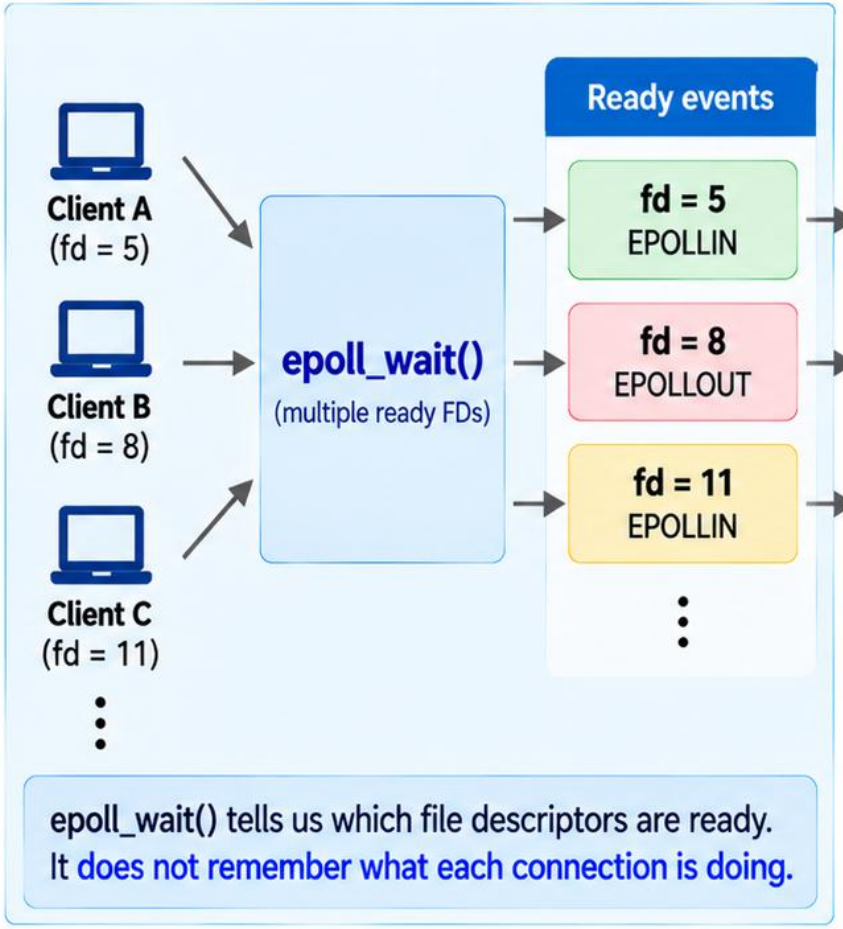


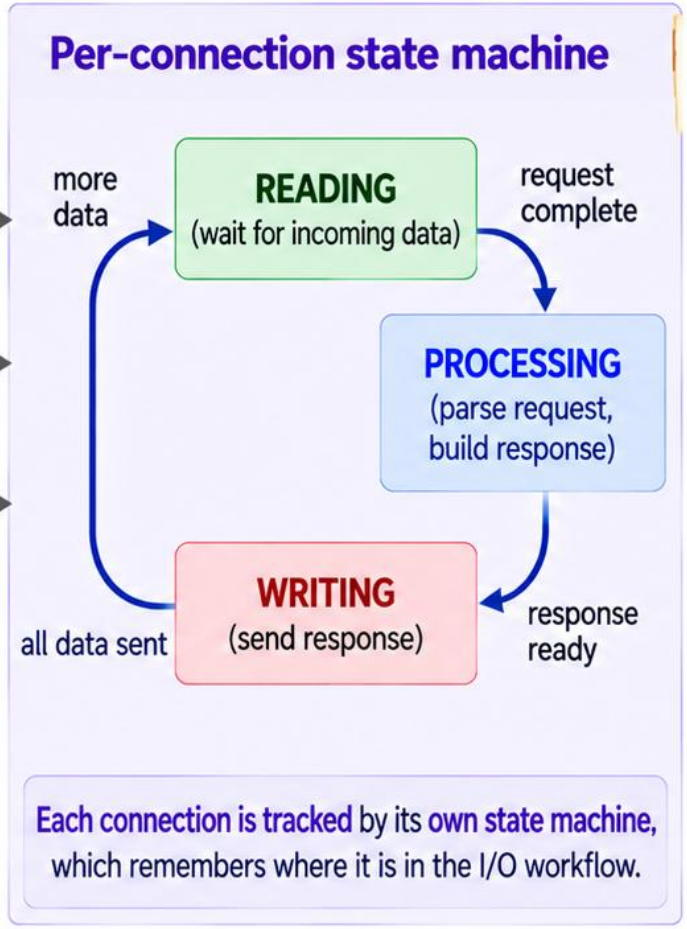






epoll-Based Server with Per-Connection State Machine





Why a state machine is needed in an epoll-based server

1. **Non-blocking I/O is partial**
   read() / write() may not complete in one call. We must remember how much has been done.
2. **Different events at different times**
   A connection can be readable, then writable, then readable again.
3. **Many connections concurrently**
   A single thread handles thousands of connections. We need per-connection context to manage them independently.
4. **Correctness and performance**
   Avoid losing events, handle EAGAIN properly, and continue from the right state.

Key Takeaway

In an epoll-based server, a **per-connection state machine is essential** to correctly and efficiently handle non-blocking I/O for many clients.

How to Implement State Machine in C

1 Per-connection data structure

```
enum conn_state {
    ST_READING,
    ST_PROCESSING,
    ST_WRITING
};

struct connection {
    int fd;
    enum conn_state state;
    char inbuf[4096];
    size_t in_len;
    char outbuf[4096];
    size_t out_len;
    size_t out_off;
    // ... other fields (e.g., address, timers)
};
```

Which I/O step this connection is at now

Input buffer and received data length

Output buffer and send progress

2 Register a new connection

```
struct connection *conn = malloc(sizeof(*conn));
conn->fd = client_fd;
conn->state = ST_READING;
conn->in_len = 0;
conn->out_len = 0;
conn->out_off = 0;

struct epoll_event ev;
ev.events = EPOLLIN;        // start with read
ev.data.ptr = conn;         // attach connection
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);
```

3 Event loop

```
struct epoll_event events[MAX_EVENTS];
int n = epoll_wait(epfd, events, MAX_EVENTS, -1);

for (int i = 0; i < n; i++) {
    struct connection *conn = events[i].data.ptr;
    uint32_t ev = events[i].events;

    if (ev & EPOLLIN)
        handle_read(conn);
    if (ev & EPOLLOUT)
        handle_write(conn);
    if (ev & (EPOLLERR | EPOLLHUP))
        close_connection(conn);
}
```

4 Handle read event (ST_READING)

```
void handle_read(struct connection *conn) {
    ssize_t n = read(conn->fd, conn->inbuf + conn->in_len,
                     sizeof(conn->inbuf) - conn->in_len);
    if (n > 0) {
        conn->in_len += n;
        if (request_complete(conn)) {
            build_response(conn);
            conn->state = ST_WRITING;
            modify_epoll(conn, EPOLLOUT);
        }
    } else if (n == 0) {
        close_connection(conn);   // client closed
    } else if (errno != EAGAIN) {
        close_connection(conn);
    }
}
```

Read available data. If a full request is received, build a response and switch to WRITING.

5 Handle write event (ST_WRITING)

```
void handle_write(struct connection *conn) {
    ssize_t n = write(conn->fd, conn->outbuf + conn->out_off,
                      conn->out_len - conn->out_off);
    if (n > 0) {
        conn->out_off += n;
        if (conn->out_off == conn->out_len) {
            conn->out_len = 0;
            conn->out_off = 0;
            conn->in_len = 0;
            conn->state = ST_READING;
            modify_epoll(conn, EPOLLIN);
        }
    } else if (n < 0 && errno != EAGAIN) {
        close_connection(conn);
    }
}
```

Send pending data. When all data is sent, switch back to READING.

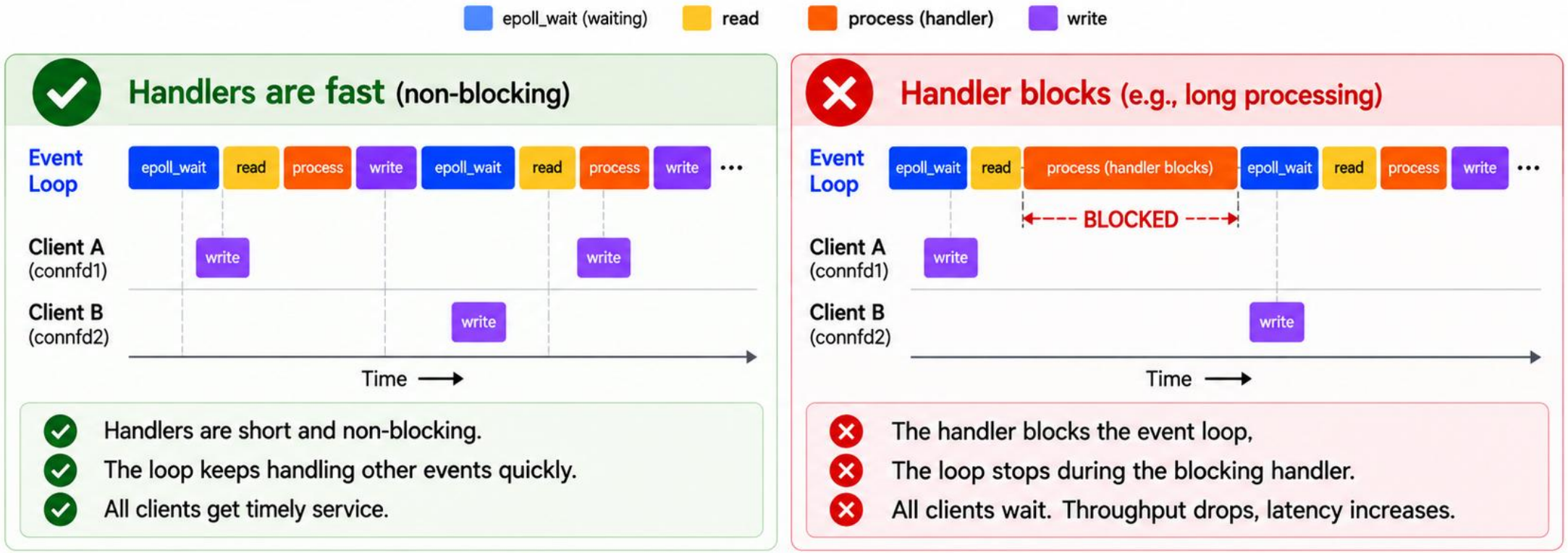
6 Modify epoll events

```
void modify_epoll(struct connection *conn, uint32_t events) {
    struct epoll_event ev;
    ev.events = events;
    ev.data.ptr = conn;
    epoll_ctl(epfd, EPOLL_CTL_MOD, conn->fd, &ev);
}
```

Key Point

Each connection keeps its own state and buffers, and the event loop dispatches events to the correct connection handler.

Event Loop Can Still Block



Why it happens

- **CPU-heavy computation**
  (e.g., image processing, ML inference)
- **Blocking I/O**
  (disk, file, DNS, DB query, network call)
- **Synchronous library calls**
  (e.g., sleep(), getaddrinfo(), malloc lock)

Effect

- No other events are handled.
- Other clients wait.
- Throughput drops, latency increases.
- A single slow task can impact all clients.

How to prevent

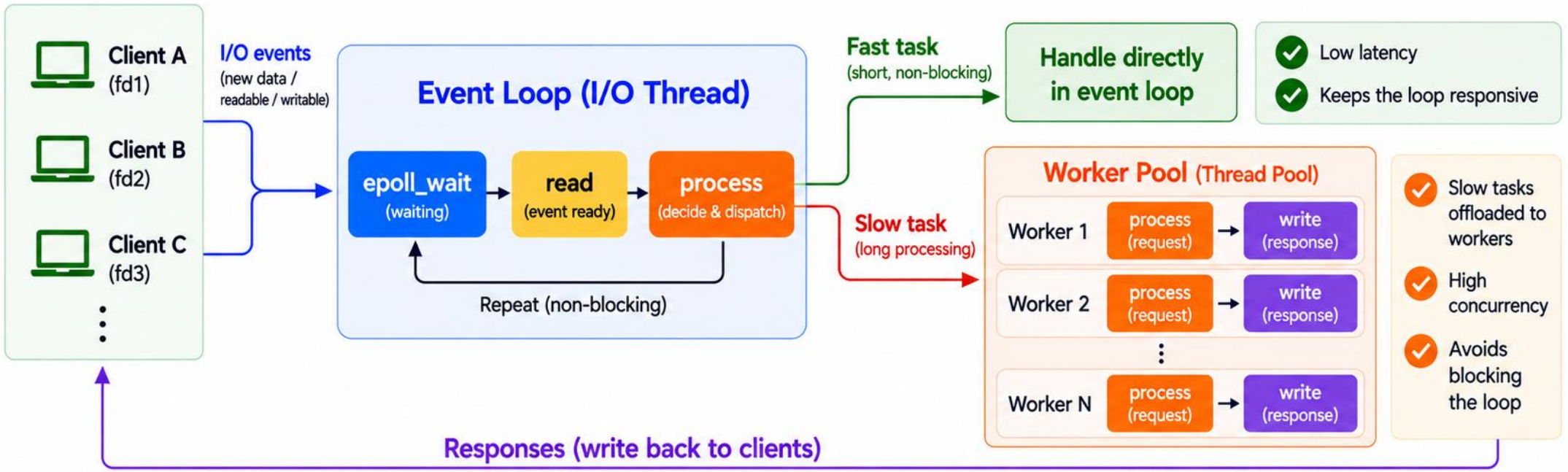
- Offload long tasks to worker threads or a thread pool / task queue.
- Use non-blocking I/O or async APIs.
- Use timerfd, eventfd, signalfd to avoid blocking waits.
- Keep event loop handlers short and non-blocking.

Event Loop

Dispatch work to workers

Worker Pool

Hybrid Approach: Event Loop with Worker Pool



How it works

1. Wait for events (epoll_wait).
2. Read event data.
3. Process fast tasks or dispatch slow tasks to worker pool.
4. Workers process and write back.
5. Repeat (non-blocking).

Benefits

- One thread handles many clients.
- Fast tasks: low latency.
- Slow tasks: offloaded to workers.
- High concurrency and scalability.
- Event loop remains responsive.

Key Points

- Keep loop handlers short and non-blocking.
- Long processing in the loop blocks everyone.
- Use a bounded queue / thread pool to avoid overload.

Color Legend

- epoll_wait (waiting)
- read (event ready)
- process (handler)
- write (response)

Observing Event Loop

1 Find CPU hotspots with perf

See which function consumes CPU time and find a long-running handler in the event loop.

```
$ sudo perf top -p 12345

Samples     %       Symbol
  3521    48.3%     process_request
  1204    16.5%     parse_json
   832    11.4%     do_computation
   421     5.8%     memcpy
   310     4.3%     __GI___libc_send
...
```

- Shows which function consumes CPU time.
- Quickly find a long-running handler in the event loop.
- Use `perf record` + `perf report` for details (call graph, source line, etc.).

2 Measure slow calls with strace

See how long each syscall takes and identify slow read/write, disk I/O, or other blocking calls.

```
$ strace -tt -T -p 12345
...
12:34:56.123456  epoll_wait(3, ..., -1) = 1 <0.000123>
12:34:56.123789  read(5, ..., 4096) = 4096 <0.000231>
12:34:56.124001  write(6, ..., 4096) = 4096 <0.250123> <-- slow
...
```

- `-T` shows how long each syscall takes.
- Identify slow read/write, disk I/O, or other blocking operations.
- Combine with `-e` to focus on specific syscalls (e.g., `-e read,write,epoll_wait`).

3 Check scheduling and wake-ups

See when the thread runs, sleeps, and is woken up. Large gaps indicate the event loop was blocked.

```
$ sudo perf sched timehist -p 12345

TIME (ms)  COMM        PID    STATE     EVENT
  0.000    event_loop  12345  RUNNING   -
  5.123    event_loop  12345  SLEEPING  ep_poll_wait
305.789    event_loop  12345  RUNNING   sched_wakeup
305.900    event_loop  12345  RUNNING   -
...
```

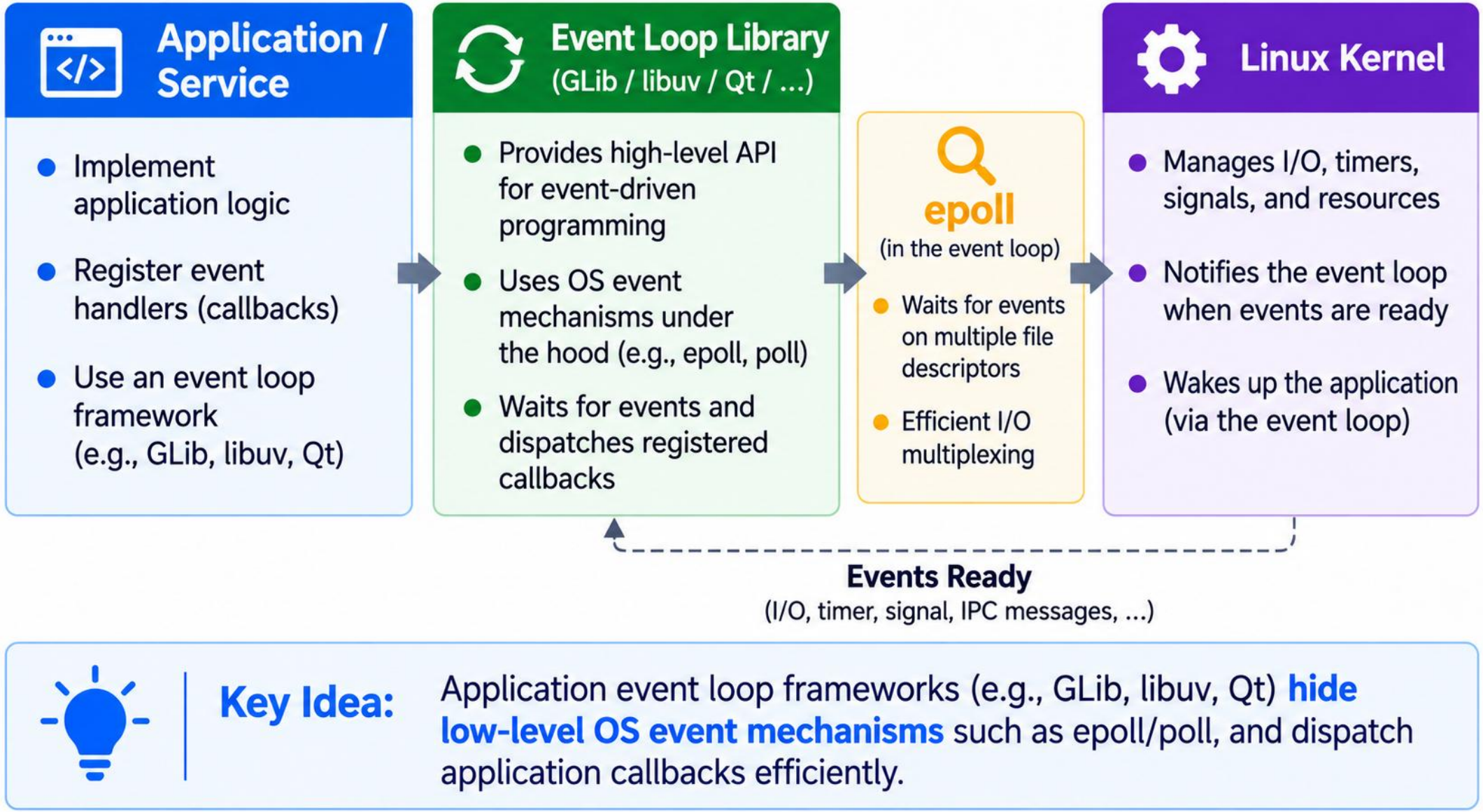
- See when the thread runs, sleeps, and is woken up.
- Large gaps indicate the thread was blocked (e.g., in a slow handler or I/O).
- Helps distinguish CPU-bound vs. I/O/blocking delays.

PART 1: Linux Execution Model

- **Week 3**

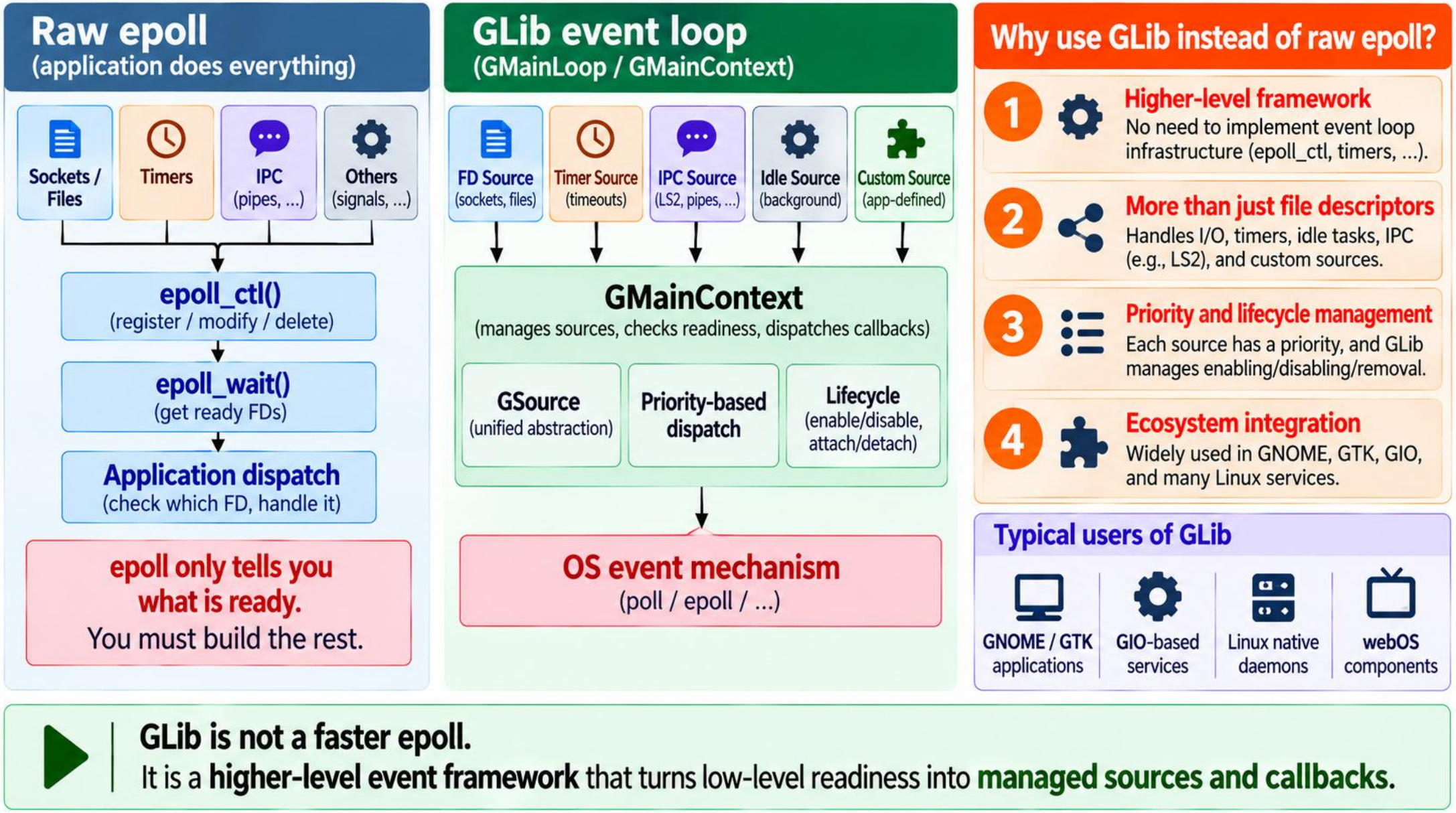
- **How to Handle Multiple Events**
  - How can a process efficiently wait for multiple events?
- **How to Handle Multiple Clients (Concurrent Servers)**
  - How can a server efficiently handle many concurrent clients?
- **Event Driven Applications and IPC in webOS**
  - How can a webOS application handle many events and communicate with other processes efficiently?

From epoll to Application Event Loops

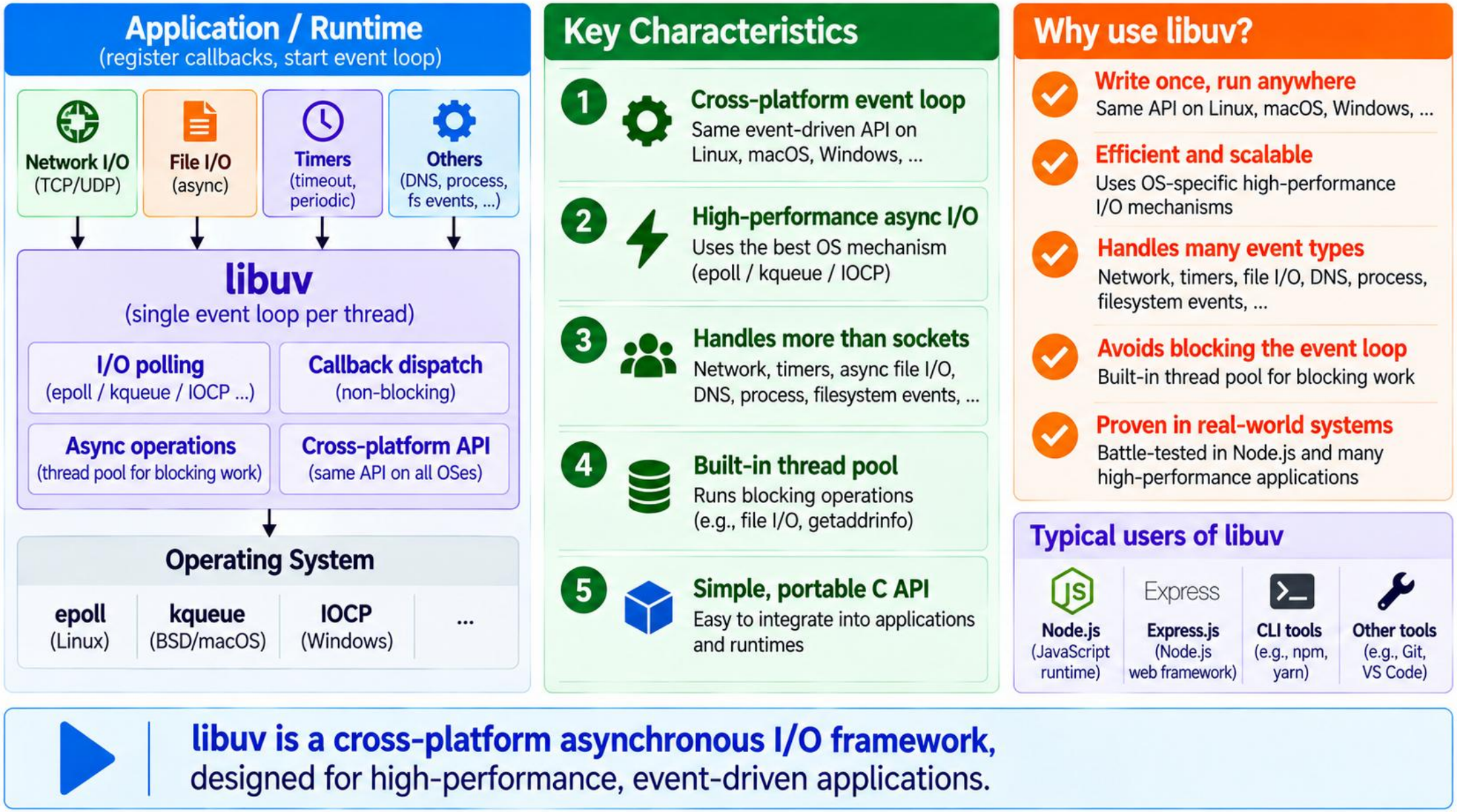


Key Idea: Application event loop frameworks (e.g., GLib, libuv, Qt) **hide low-level OS event mechanisms** such as epoll/poll, and dispatch application callbacks efficiently.

Glib Event Loop



Libuv Event Loop



Event Loop in the webOS

Native Services (C / C++)

e.g., **SAM**, system services, media service, audio manager, power manager, etc.

GLib GMainLoop

- Common pattern in webOS native services (including **SAM**)
- Event-driven callbacks (I/O, timers, signals, etc.)
- Integrates with LS2 and other libraries

Node.js Services (JavaScript)

e.g., webOS JS services, app services, tooling, development utilities

libuv Event Loop

- Node.js runtime uses libuv event loop
- Handles async I/O, timers, child processes, etc.
- Widely used for webOS application services

LSM / System UI (Qt / QML)

e.g., Home Launcher (LSM), TV UI, settings UI

Qt Event Loop (QEventLoop)

- LSM is implemented with Qt / QML
- Event-driven UI, input, timers, IPC, etc.
- Integrates with Linux event mechanisms under the hood

Web Runtime / Web Apps (Chromium)

e.g., WAM (Web App Manager), web apps (HTML/JS)

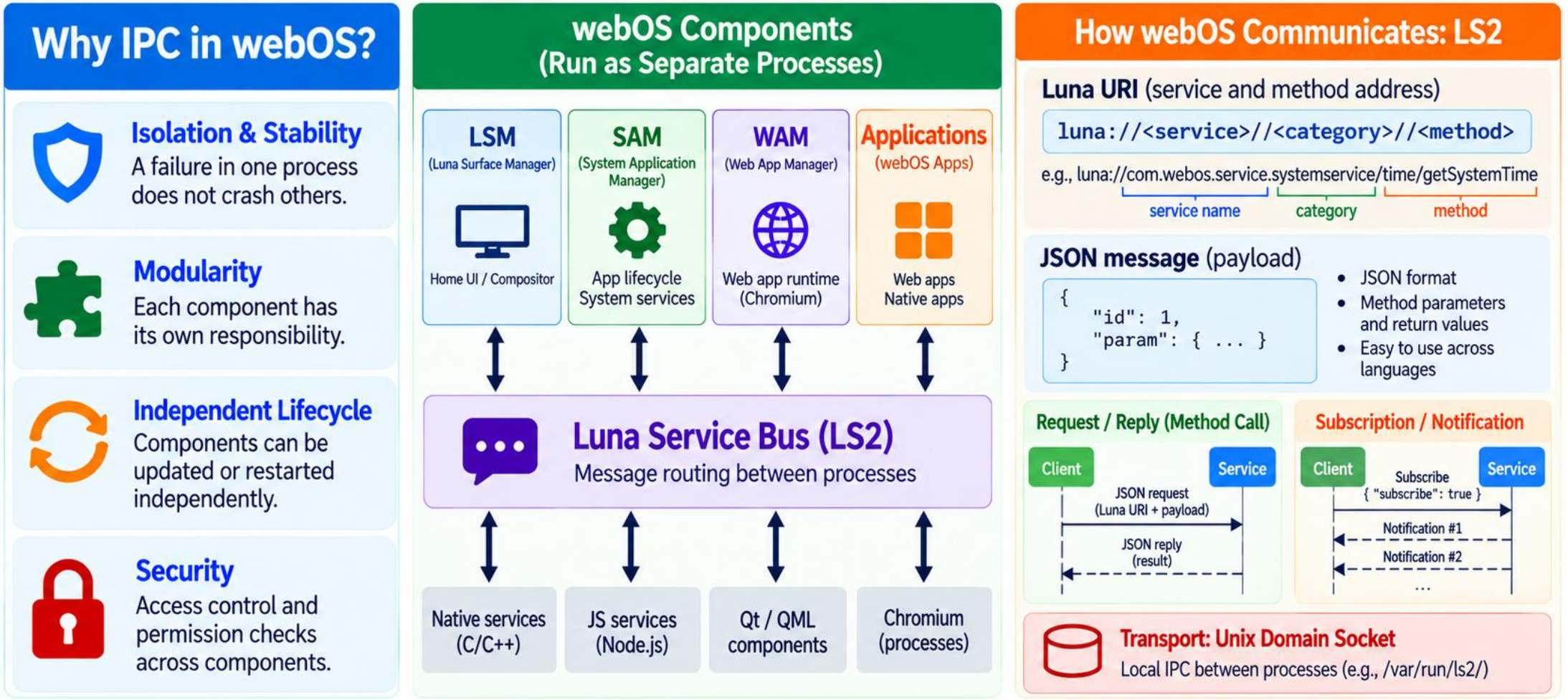
Chromium Event Loop

- Chromium multi-process architecture (browser, renderer, GPU, etc.)
- Event-driven I/O, timers, user input, IPC, etc.
- Built on platform event mechanisms (e.g., epoll/poll)

Key Takeaways

1. Different webOS components use **different event-loop frameworks** (GLib, libuv, Qt, Chromium, etc.) suited to their purpose.
2. These frameworks provide higher-level APIs and ultimately rely on **Linux event mechanisms** (e.g., epoll/poll) underneath.

Why and How Do webOS Processes Use IPC?

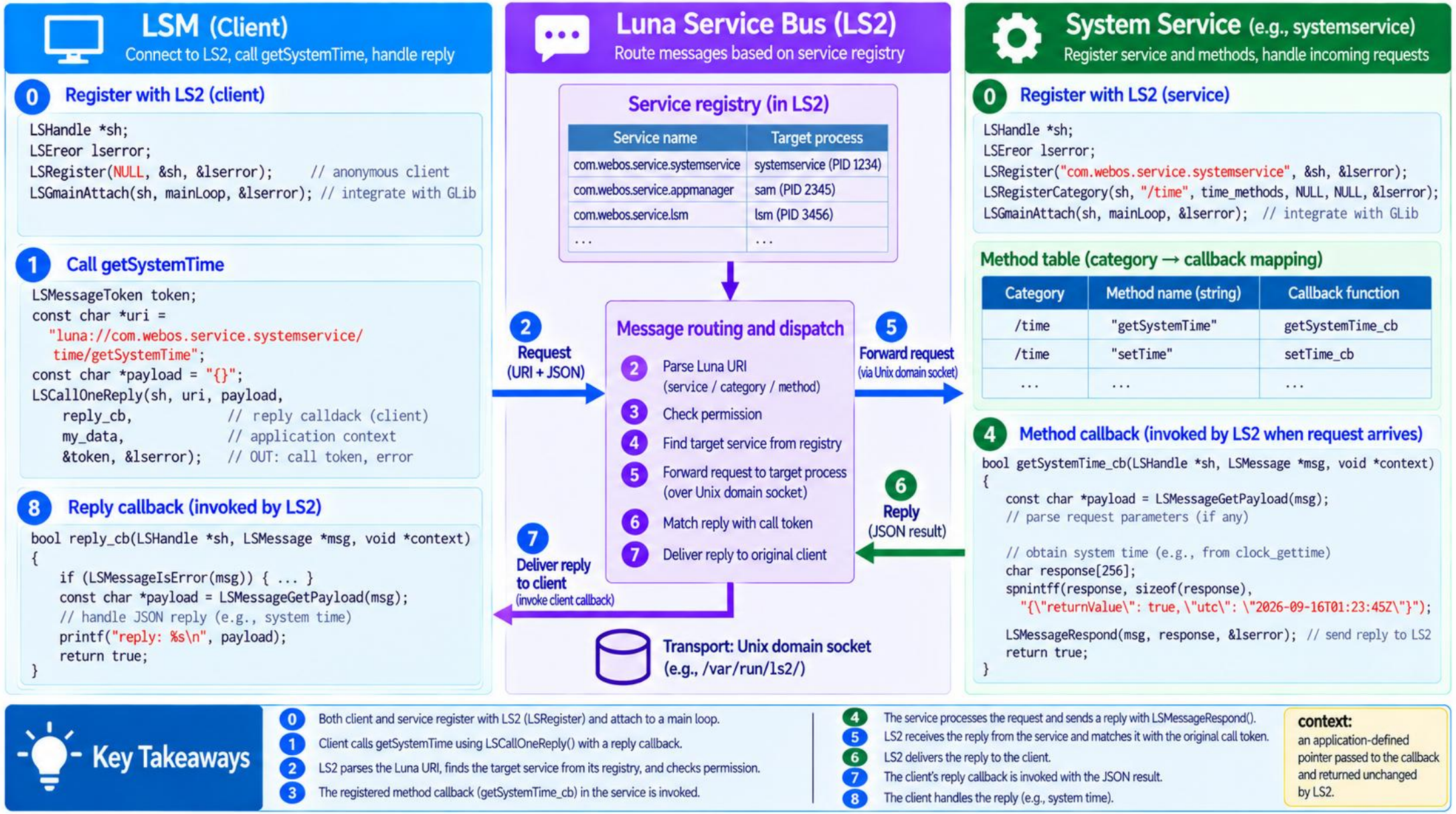


Key Takeaway

webOS separates system functions into independent processes and uses Luna Service Bus (LS2) for asynchronous, message-based IPC.

Luna URI identifies the target service and method | JSON messages carry the payload | Supports request/reply and subscription | Uses Unix domain socket

Scenario: LSM Calls getSystemTime Method via LS2



Wayland: Control Messages and Shared Buffers

Why ordinary IPC is not enough

Graphics data is large and high frequency



1920 × 1080 × 4 bytes
≈ **8 MB** / frame
60 FPS
≈ **500 MB/s**
(raw pixel data)

Sending graphics via ordinary IPC

- Copy pixel data into each message
- High CPU and memory bandwidth cost
- Increases latency
- Not scalable for high frame rate
- Cannot easily share buffers or synchronize rendering

Wayland solves this with a split design

Control messages (small, infrequent)

- create_surface
- attach (buffer)
- commit
- configure (from compositor)
- input events
- ...

Unix Domain Socket (Wayland protocol)

- Low latency
- Small messages
- Bidirectional

Graphics buffers (large, frequent)



- Render into shared buffer (wl_shm or dma-buf)
- No data copy through socket
- Compositor imports buffer
- Synchronize using fences/events
- Supports zero-copy (dma-buf)

Shared Memory Buffer (wl_shm / dma-buf)

- Share buffer between client and compositor
- Avoid copying large pixel data
- Efficient and scalable for high FPS

Benefits of Wayland approach

Efficient data transfer

- Large pixel data is shared, not copied
- Low CPU and memory overhead

Smooth and scalable rendering

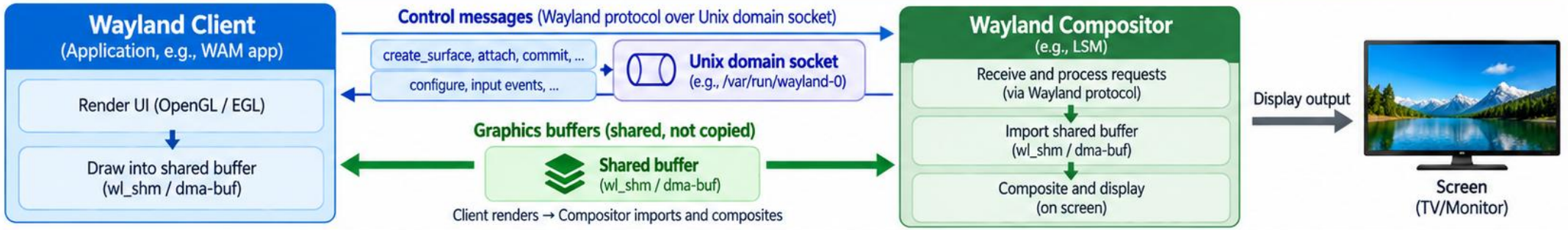
- Supports high resolution and high FPS
- Suitable for modern UI/graphics

Better synchronization

- Buffer ownership and lifecycle management
- Explicit synchronization (fences, events)

Flexible buffer types

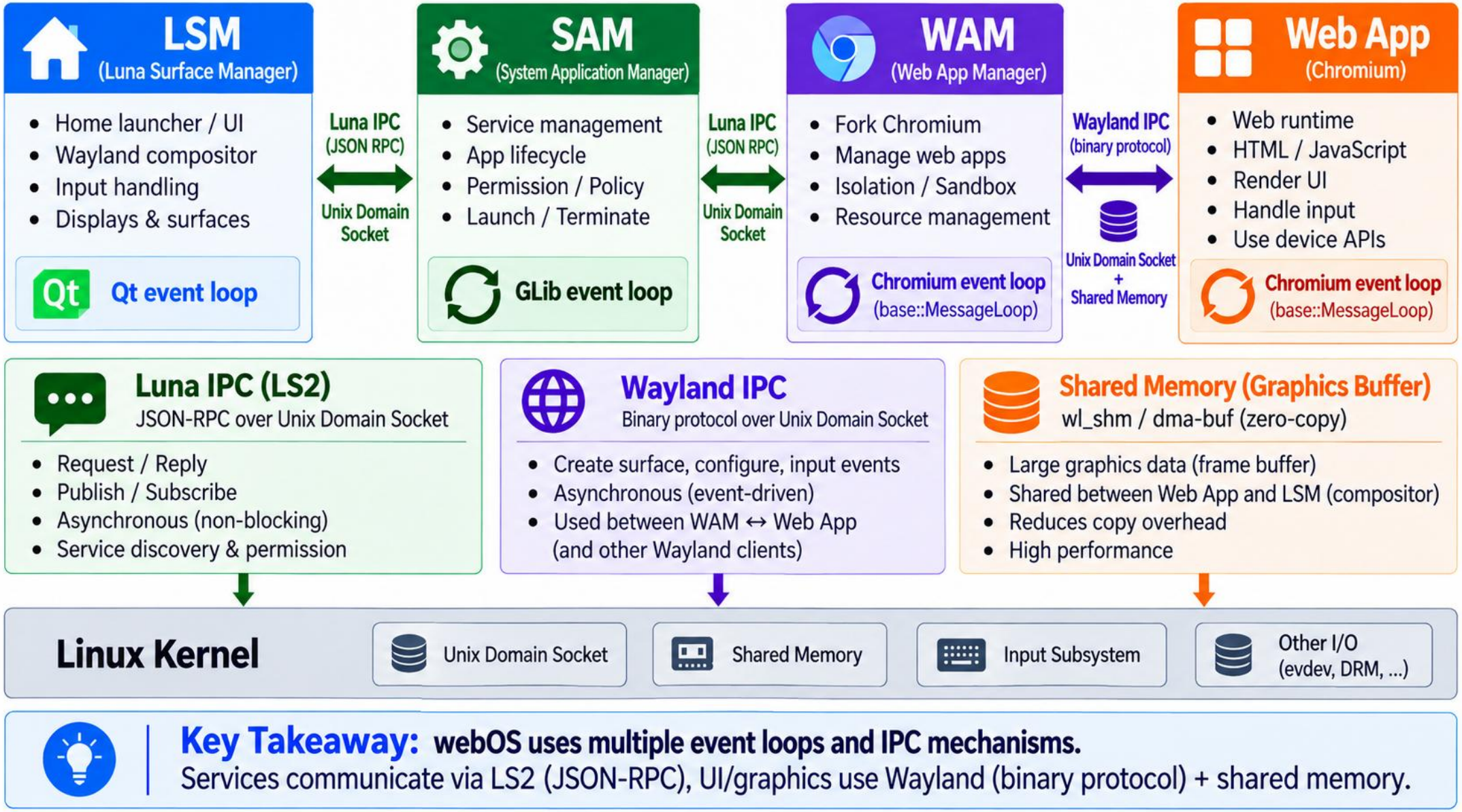
- wl_shm (shared memory)
- dma-buf (zero-copy, GPU buffers)
- Works across different hardware



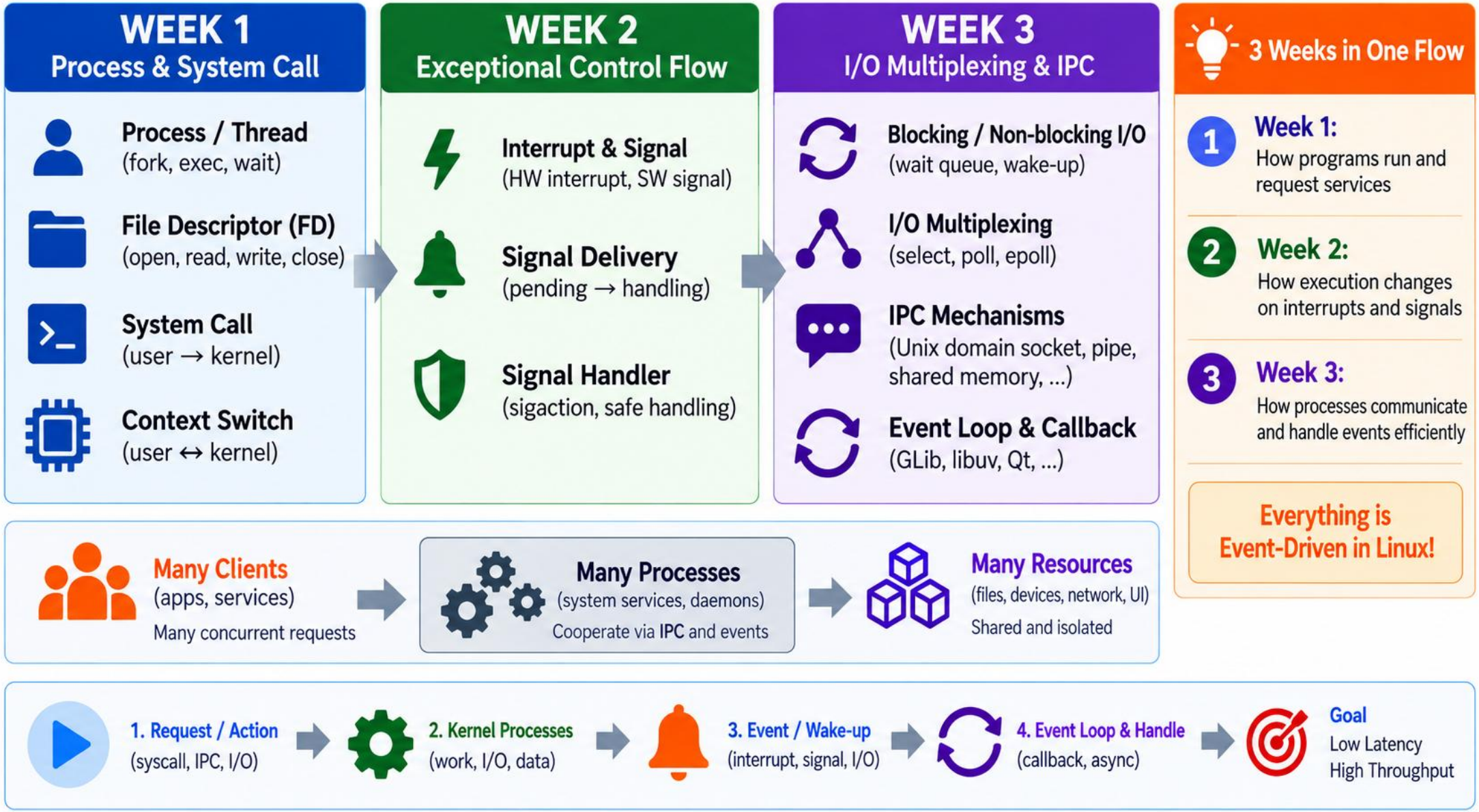
Key Takeaways

- **Ordinary message-based IPC** is **inefficient** for large, high-frequency graphics data.
- **Wayland** is a protocol between applications and the compositor that **separates control messages** from graphics buffers.
- **Control messages** (e.g., create_surface, attach, commit) are sent over a **Unix domain socket**.
- **Graphics buffers** are shared via wl_shm or dma-buf, avoiding data copies and enabling efficient, scalable rendering.

Putting It Together: webOS Event Driven Architecture



Summary: From Basic to webOS Event-Driven Systems



End of Week 3

Sungyong Park, Ph.D. | parksy@sogang.ac.kr



